

# CASSANDRA: Consensus with Partial Progress via Robust Partitionable View Synchronization

Shaokang Xie  
University of California, Davis

Dakai Kang  
University of California, Davis

Junchao Chen  
University of California, Davis

Suyash Gupta  
University of Oregon

Daniel P. Hughes\*  
Radix DLT Ltd.

Mohammad Sadoghi  
University of California, Davis

## Abstract

Replicated databases and permissioned blockchains rely on Byzantine Fault-Tolerant (BFT) consensus to maintain a consistent transaction order across replicas. These protocols preserve safety even under asynchrony, as they commit a transaction only after agreement among a strong quorum of replicas. During network partitions, however, when no strong quorum is reachable, they lose liveness and cannot make useful progress.

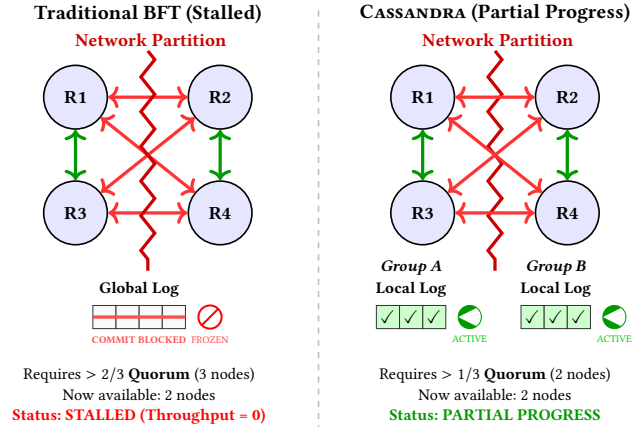
In this paper, we present CASSANDRA, a consensus protocol that enables partial progress without sacrificing safety. CASSANDRA achieves this through a two-tier certification framework that decouples availability from commitment, allowing each partition to extend its own chain and reconcile these chains once the network is restored. To support this, CASSANDRA introduces a pacemaker that advances views without requiring a strong quorum and calibrates each replica’s timeout off the critical path.

Our evaluation results show that CASSANDRA remains competitive with state-of-the-art BFT protocols under stable conditions, sustaining 900K TPS at 16 replicas and 480K TPS at 104 replicas, with latency ranging from 0.31s at 16 replicas to 0.75s at 104 replicas. Under severe partitions, CASSANDRA maintains non-zero speculative throughput through PoA-backed progress, preserving work that can be reconciled once connectivity is restored.

## 1 INTRODUCTION

Decentralized systems offer their clients an immutable ledger even in the presence of faulty or Byzantine behavior. To achieve this, they employ a Byzantine Fault-Tolerant (BFT) consensus protocol that allows multiple distributed parties to maintain a single, globally consistent transaction order over a shared state [21, 45]. Because these parties often hold the same data, they are termed replicas. State-of-the-art BFT protocols, which have served as the foundational building blocks of permissioned blockchains, provide strong fault tolerance and achieve high throughput and low latency under stable network conditions [10, 52].

In practice, however, networks are not always stable. Transient network partitions, delay asymmetries, and intermittent connectivity can disrupt communication among replicas. High-impact production incidents reported by major technology providers such as Amazon and Cloudflare demonstrate that network partitions are not merely a theoretical concern but a recurring operational reality with severe consequences [3, 4, 11–13]. For example, a major Amazon EC2/EBS outage in the us-east-1 region revealed how misrouted traffic and aggressive recovery storms can effectively



**Figure 1: Comparison between traditional BFT protocols and CASSANDRA under a network partition. In a 2-2 partition scenario, traditional protocols stall as they cannot reach the required 2/3 quorum, whereas CASSANDRA maintains partial progress using its Proof of Availability (PoA) mechanism.**

partition storage clusters, rendering data unavailable for extended periods and causing widespread downtime across SaaS providers and financial platforms, with aggregate losses reaching millions of dollars [4]. Similarly, Cloudflare has experienced outages in which routing anomalies, overloaded or impaired interconnects, and regional reachability failures created partitions between cloud regions [11, 12]. The impact is even more pronounced in decentralized and blockchain systems: network congestion-induced outages in Solana have led to millions of dollars in losses for individual traders [13]. Moreover, partitions are not always accidental: routing-layer attacks, such as BGP hijacks, have been shown to isolate nodes and split the Bitcoin network into disconnected components [6]. These incidents highlight partition-like failures as a practical risk for geo-distributed services.

For BFT replicated systems, such partition-like failures create a quorum-stall problem: protocols preserve *safety*—that is, no two correct replicas commit conflicting transactions—but fail to process new requests when no connected component contains a sufficiently large quorum. This tension between safety and availability is a consequence of the CAP theorem (*Consistency, Availability, and Partition tolerance*), which states that in the presence of network partitions, a distributed system cannot simultaneously guarantee both consistency and availability [8, 18]. Consequently, BFT systems designed for mostly reliable networks can lose availability under partitions, even though they continue to preserve safety.

\*In loving memory of Daniel P. Hughes, a valued friend and colleague, whose foundational contributions, ideas, and passion continue to inspire and shape this work.

BFT-based decentralized systems can stall under network partitions because the underlying consensus protocol suffers from the following structural dependencies:

**C1: Strong-Quorum Dependence.** Traditional BFT protocols operate under the assumption that the system has at least  $n = 3f + 1$  replicas, where  $f$  denotes the number of faulty replicas. To guarantee that all correct replicas agree on a single transaction order, these protocols require a *strong quorum*—at least  $n - f$  ( $2f + 1$ )—of replicas to certify a proposal. Under a network partition, replicas are divided into multiple *connected components* and replicas in different components are unable to communicate with each other (refer to Fig. 1). If no connected component contains a strong quorum (has fewer than  $n - f$  replicas), then no proposal can form a certificate that justifies non-conflicting commitment.

**C2: Designated-Leader Dependence.** Classic BFT protocols (such as PBFT [9] and HotStuff [52]) operate in rounds, each with a designated leader responsible for determining the order of transactions. However, under a network partition, the leader is unreachable for all but the replicas in its connected component. When progress stalls, replicas assume that the leader has failed and initiate the recovery protocol to replace the leader. However, the recovery protocol itself requires participation of a quorum of replicas. If no partition contains a strong quorum, replicas repeatedly attempt leader replacement without ever making progress.

**C3: Round-Overlap under Varying Network Delay.** Certificate formation also requires replicas to remain in the same round long enough to exchange proposals and votes. Thus, progress depends on sufficient *round overlap* among replicas in the same connected component. However, under partial synchrony, the effective delay bound is unknown before stabilization and may vary across stabilized periods [9, 16]. A timeout that is too small may cause replicas to leave a round before messages arrive, while an overly conservative timeout increases latency.

**Consequence: Zero Useful Progress.** Together, strong-quorum dependence, leader dependence, and round-overlap requirements imply that existing BFT protocols make no useful progress during a partition unless some connected component can both gather the required quorum and keep enough replicas synchronized in the same round. Replicas may continue retransmitting messages and triggering recovery attempts, but no new transactions can be committed, resulting in zero throughput. [This limitation is shared by performance-oriented BFT protocols, although they optimize different aspects of the good case. Concurrent and multi-leader designs \[7, 22, 29, 38–40, 47, 48\] improve throughput; the HotStuff family \[28, 41, 52\] pipelines or shortens the commit path; DAG or dissemination-decoupled protocols \[14, 19, 31, 46\] improve throughput by separating data dissemination from ordering; and optimistic fast-path protocols \[20, 34, 42\] reduce latency through a fast certification path. Despite these optimizations, without reaching a strong quorum, their progress towards ordering or finality stalls.](#)

In this paper, we present **CASSANDRA**, a novel consensus protocol that revisits the practical implications of CAP in decentralized replicated systems [21]. Rather than attempting to achieve full liveness during partitions, CASSANDRA enables *Partial Liveness*, allowing *partial progress* while strictly preserving safety during network

partitions. CASSANDRA addresses the above challenges through three complementary mechanisms. First, CASSANDRA introduces a *two-tier certification mechanism* for each transaction:

- *Proof of Availability* (PoA), a certificate formed by collecting votes from a weak quorum of  $f + 1$  replicas.
- *Proof of Reliability* (PoR), a certificate formed by collecting votes from a strong quorum of  $n - f$  replicas.

A PoA certificate enables replicas to make *partial progress* during a network partition. However, to guarantee consistency, CASSANDRA requires PoR certificates to commit transactions.

Second, CASSANDRA does not designate any replica as “the leader”; instead, it allows all replicas to propose transactions. Each replica then applies our deterministic *proposal priority rule* to independently select the *strongest proposal* among those it has received. As a result, even during a network partition, every replica can always identify a locally strongest proposal to vote for.

Third, CASSANDRA deploys a novel **decoupled pacemaker** for round or view synchronization. Traditional pacemakers do perform round synchronization, but they face the following challenge:

**C4: Pacemaker Constraints.** Existing pacemakers guarantee sufficient round overlap among a strong quorum of  $2f + 1$  replicas, which may be unavailable under network partitions. Achieving this requires the pacemaker to perform two tasks: advancing replicas’ rounds (or views), and calibrating their timers so that these rounds overlap. However, the pacemaker performs both tasks on the critical consensus path, which degrades system performance.

CASSANDRA decouples these tasks as follows: logical round advancement remains certificate-driven on the critical consensus path, while timeout calibration runs in the background—each replica adjusts its local round timeout independently, without blocking consensus progress. However, one challenge still remains unresolved.

**C5: Convergence.** Although PoA certificates enable progress under network partitions, they also allow different connected components to advance along distinct PoA-backed branches. Because replicas can observe different subsets of proposals, conflicting PoA certificates can coexist, and Byzantine replicas may further equivocate across competing branches. Consequently, once the network is restored, CASSANDRA must ensure that these divergent partial histories can be reconciled into a single committed order.

CASSANDRA addresses this challenge in two ways. First, its deterministic proposal-priority rule makes reconciliation *implicit*: once connectivity is restored, correct replicas identify the highest-priority branch as the strongest, vote for it, and extend it with new PoRs. Second, the *decoupled pacemaker* enables lagging replicas to catch up whenever they observe higher-round certificates. As a result, once a strong quorum becomes reachable again, all correct replicas—whether previously stalled or progressing within separate partitioned components—reconcile their histories by extending the same strongest branch.

Our evaluation demonstrates that CASSANDRA sustains partial progress during network partitions while remaining competitive under stable network conditions. Under steady-state, CASSANDRA achieves 900K TPS at 16 replicas and 480K TPS at 104 replicas, retaining 53.6% of its throughput, while its latency increases from 0.31s at 16 replicas to 0.75s at 104 replicas. During network partitions, CASSANDRA achieves a ‘0-to-1’ breakthrough by sustaining

*Continuous PoA Generation* with a speculative throughput of 500K TPS even when global consensus stalls (Figure 12.c). This accumulated partial progress enables a rapid *recovery burst* upon network restoration, allowing the system to efficiently reconcile divergent histories and resume peak performance quickly.

Specifically, we make the following contributions:

- We introduce a two-tier certification mechanism that separates partial progress (PoA) from final commitment (PoR), enabling progress within connected components while preserving safety.
- We eliminate reliance on a designated leader by allowing all replicas to propose continuously and to independently vote for the strongest proposal according to a deterministic proposal-priority rule. This design enables local chain extension during partitions.
- We design a decoupled pacemaker that enables round advancement even with weak quorums, while calibrating local timeouts in the background, off the critical path.
- We show that divergent PoA-backed histories accumulated during partitions can reconcile once connectivity is restored.
- We conduct an extensive evaluation of CASSANDRA under both stable and partitioned network conditions. Our results demonstrate that CASSANDRA remains competitive with state-of-the-art protocols in synchronous settings while sustaining non-zero useful progress during severe partitions.

## 2 SYSTEM MODEL

**Adversary Model.** We consider a distributed system consisting of  $n$  replicas, each assigned a unique identifier  $i \in \{1, \dots, n\}$ . We denote the replica with identifier  $i$  by  $p_i$ . The system tolerates up to  $f$  Byzantine replicas, where  $n \geq 3f + 1$ . Byzantine replicas, also referred to as faulty replicas, may exhibit arbitrary or malicious behaviors that deviate from the consensus protocol. The remaining replicas are *correct* (or *honest*) and strictly follow the protocol. The system processes requests from clients, and we allow any number of clients to be arbitrarily faulty.

**Authenticated Communication.** We assume an authenticated communication model based on standard public-key cryptography [30]. Each client and replica possesses a public-private key pair and uses digital signatures to authenticate messages. A sender signs a message using its private key, and any recipient can verify the signature using the corresponding public key. We denote by  $\langle m \rangle_{p_i}$  a digital signature over message  $m$  produced by replica  $p_i$ .

Correct replicas accept only well-formed messages that carry valid signatures. We assume that Byzantine replicas are computationally bounded and cannot forge signatures or break the cryptographic primitives used by correct replicas.

We further assume the existence of a collision-resistant hash function  $H(\cdot)$ , meaning it is computationally infeasible to find two distinct inputs  $x \neq y$  such that  $H(x) = H(y)$ .

**Threshold-Signature Setup.** In addition to ordinary digital signatures, CASSANDRA uses threshold signatures for compact certificates: an  $(f+1, n)$  scheme for weak-quorum certificates and the deterministic threshold coin and a  $(2f+1, n)$  scheme for strong-quorum PoR certificates. We assume that the corresponding threshold verification keys and replica signing shares are generated before any runtime partition, either by a trusted dealer or by executing a standard DKG protocol [17, 44] while the replicas are fully connected.

**Network Model.** We assume the *partial synchrony* model of Dwork *et al.* [16]. Existing BFT protocols assume the existence of a single Global Stabilization Time (GST) after which the network remains permanently synchronous [28, 52]. In contrast, we incorporate a more realistic setting that allows recurring network partitions. Specifically, we assume an unbounded sequence of stabilized periods, each beginning at a stabilization point  $GST_k$ , where  $GST_1 < GST_2 < \dots$ . During each stabilized period, the network is synchronous for a sufficiently long but possibly finite interval, meaning that messages between correct replicas are delivered within an *unknown* bound  $\Delta$ . Outside such synchronous periods, we make no global timing or connectivity assumptions: the network may be asynchronous, partially connected, or partitioned into multiple connected components. Our partial-progress guarantees are therefore conditioned on any connected component in which messages among mutually reachable correct replicas are delivered within  $\Delta$ .

## 3 ACHIEVING PARTIAL PROGRESS UNDER NETWORK PARTITIONS

Traditional BFT protocols cannot make progress during a network partition unless at least  $2f+1$  replicas can reach one another; we refer to such a set as a *strongly connected component*. CASSANDRA, by contrast, aims to enable *partial progress* while preserving safety even when no such strongly connected component exists. It achieves this goal through the following design principles:

(1) **Eliminating futile recovery traffic.** When no strongly connected component exists, it is wasteful for replicas to repeatedly trigger recovery mechanisms in an attempt to achieve global synchronization. Instead, they should continue issuing new proposals based on locally available connectivity.

(2) **Partitionable leaders.** During a network partition, not all replicas can necessarily reach the replica designated as leader. Instead, each replica should take over this role itself, independently selecting the strongest proposal from those it has received.

(3) **Weak-quorum ordering.** During a network partition, replicas may be unable to reach a strongly connected component. In that case, they should order transactions based on *weak quorums*, while recognizing that such decisions are non-final and must be reconciled once a strong quorum becomes available.

(4) **Decoupled pacemaker.** Traditional pacemakers are designed to synchronize a strong quorum of replicas, and they do so by placing all round-synchronization logic on the critical consensus path. Because such a quorum may be unavailable during a network partition, a pacemaker should instead be able to coordinate a weak quorum of replicas, while keeping much of this coordination off the critical path.

### 3.1 Tools for Partial Progress

To implement the aforementioned design principles, we introduce:

- (1) *Partitionable Leader Election.*
- (2) *Decoupled Pacemaker Design.*

Next, we explain how these tools help CASSANDRA enable partial progress during network partition.

(1) **Partitionable Leader Election.** Permitting leader election to proceed even when no strongly connected component exists

allows each partition to make progress independently, without waiting for a dedicated leader. We achieve this goal through two new principles: *two-tier certification* and *deterministic proposal priority*.

**Two-Tier Certification.** Because no strongly connected component is available during a network partition, we allow replicas to make decisions based on weaker connected components. Consequently, to guarantee safety, we require two types of certificates.

- **Proof of Availability (PoA).** A PoA requires votes from at least  $f+1$  replicas and guarantees that the proposal's payload is retrievable within the system, but it does not guarantee a final commit.
- **Proof of Reliability (PoR).** A PoR requires votes from at least  $n-f$  replicas. It certifies global consistency and prevents conflicting commits.

**Deterministic Proposal Priority Rule.** Eliminating the need for replicas to wait for a designated leader means they must instead determine the next leader themselves, and this determination must be deterministic so that PoAs can be formed. Consequently, in each round, every replica broadcasts a proposal and independently identifies the leader from the proposals it receives.

Each proposal is ranked according to the recency of the certificates it carries—first by the highest referenced PoR, and then by the highest referenced PoA. Intuitively, proposals backed by more recent strong-quorum evidence are preferred, while weak-quorum evidence serves as a tie-breaking signal reflecting more recent partial progress. If multiple proposals remain tied within the same round, replicas resolve the tie using a deterministic threshold-coin mechanism derived from proposal signature shares. This guarantees that all correct replicas observing the same proposal set select the same *strongest proposal*, enabling partition-local convergence without centralized leadership.

**Implicit Reconciliation on Network Recovery.** During a network partition, replicas may form PoA certificates for diverging chains within different components. However, because proposal selection is deterministic and prioritized by certificate recency, correct replicas converge on the same strongest proposal and vote for it as soon as the network recovers. New PoRs then form on that branch, re-establishing a single, globally committed order. Reconciliation is thus not performed by an explicit merge protocol; it emerges naturally from deterministic proposal selection once certificate-driven round synchronization is restored.

**(2) Decoupled Pacemaker Design.** Although partitionable leader election is necessary for progress, it is not sufficient. A successful election requires replicas to remain in the same *round* (or *view*) long enough to exchange proposals and votes, so a mechanism for round synchronization is essential. Prior work employs a *pacemaker* that aligns a strong quorum of  $n-f$  replicas on a common round, ensuring sufficient overlap for them to exchange messages. Moreover, the pacemaker runs on the critical consensus path and thus degrades consensus performance.

We achieve this through a novel decoupled pacemaker that separates the pacemaker's two tasks: *round advancement* and *timeout calibration*. Round advancement remains on the critical consensus path, while timeout calibration is now handled by a separate background mechanism.

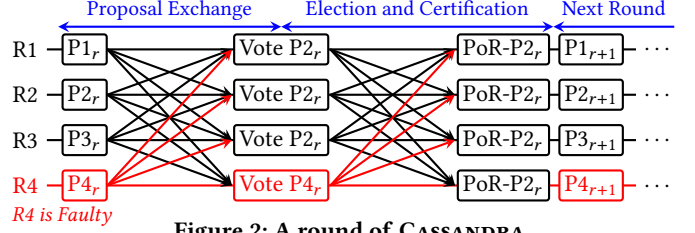


Figure 2: A round of CASSANDRA.

**Round Advancement.** A replica advances from round  $r$  to  $r+1$  under two cases: (i) when a PoR is formed, indicating successful certification by a strong quorum; or (ii) when, after round  $r$  times out, a weak quorum of  $f+1$  replicas requests a round advancement. In both cases, there is sufficient information for lagging replicas to jump immediately to the *highest certified round*. Convergence toward a common round is guaranteed because round numbers increase monotonically and replicas accept certificates only for strictly higher rounds. During synchronous periods, certificates propagate within  $\Delta$ , bounding divergence among correct replicas.

**Timeout Calibration.** Another equally important role of the pacemaker is ensuring sufficient round overlap among replicas to facilitate the exchange of messages and proposals. This must happen even under unknown and time-varying delays, and it should remain off the critical path. We meet this goal not by forcing all replicas to synchronize on a single timeout value, but by allowing each replica to calibrate its own local timeout.

### 3.2 System Guarantees

In a system of  $n = 3f+1$  replicas, where up to  $f$  replicas may be Byzantine, CASSANDRA offers the following guarantees:

- **Safety:** If two correct replicas  $p_i$  and  $p_j$  commit two transactions  $tx_1$  and  $tx_2$  at the same position in the log, then  $tx_1 = tx_2$ .
- **Liveness:** During any sufficiently long stabilized synchronous period, correct replicas continuously commit new proposals containing client transactions.

Moreover, CASSANDRA offers *partial liveness* even during network partitions. Specifically, we introduce the following property:

- **Partial Liveness:** While the network is partitioned, any synchronous connected component containing at least  $f+1$  correct replicas continues ordering new proposals containing client transactions.

## 4 CASSANDRA

CASSANDRA aims to drive BFT consensus among replicas while guaranteeing partial liveness even during periods of network partition. To this end, CASSANDRA must ensure the following:

- **Partitionable chain construction.** This enables replicas within each connected component to keep proposing, allowing the chain to grow even when a strong quorum is unreachable. CASSANDRA achieves this through *partitionable leader election*.
- **Round synchronization.** This keeps replicas sufficiently synchronized to exchange messages and form certificates, and provides lagging replicas a mechanism to catch up. CASSANDRA achieves this through the *decoupled pacemaker*.

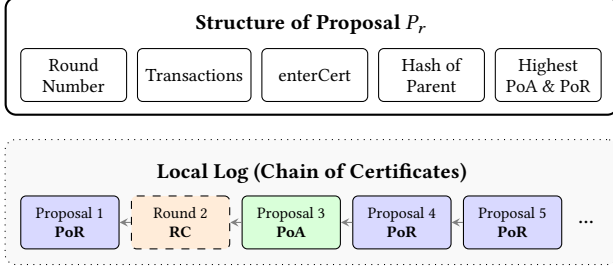


Figure 3: CASSANDRA's Proposal and Local Log Chain.

#### 4.1 Partitionable Chain Construction

CASSANDRA proceeds in a sequence of rounds (or views). Each round consists of two phases: *proposal exchange* and *election and certification* (refer to Figure 2). A replica enters a new round by following the pacemaker's round advancement rules (§3.1).

*Phase 1: Proposal Exchange.* At the start of round  $r$ , each replica aggregates a set of pending client transactions into a proposal. Each proposal *extends* the last committed proposal; each replica maintains a local log  $\mathcal{L}$  of committed proposals (Fig. 3) and sets the hash of the proposal at the tail as *parent* and its corresponding certificate as the highest PoR. Additionally, the proposal includes the highest PoA (if any) and a certificate, which proves that the sender is in round  $r$ .

This exchange takes place during an *exchange window*, in which replicas collect round- $r$  proposals from one another. As in other partially synchronous protocols, CASSANDRA assumes that each replica  $i$  maintains a local timeout parameter  $\delta_i$  (with  $\delta_i \leq \Delta$ ) that delimits the phases within each round. However, in CASSANDRA the value of  $\Delta$  is unknown a priori, so we make the value of  $\delta_i$  able to calibrate dynamically over time (Section 4.2.3). Within a connected component, we assume an exchange window of size  $2\delta_i$ , which should suffice for proposals from honest replicas to propagate under synchrony.

*Phase 2: Election and Certification.* Next, from all the valid proposals a replica receives, it deterministically selects a *Strongest Proposal* and broadcasts its VOTE. Concretely, each proposal  $\mathcal{P}$  is ranked by its latest certificates (PoR and PoA),

$$S(\mathcal{P}) \triangleq \left( \text{Round}(\mathcal{P}.\widehat{\text{PoR}}), \text{Round}(\mathcal{P}.\widehat{\text{PoA}}) \right),$$

where  $\mathcal{P}.\widehat{\text{PoR}}$  and  $\mathcal{P}.\widehat{\text{PoA}}$  denote the highest-round PoR and PoA referenced by  $\mathcal{P}$  and  $S(\mathcal{P})$  is the strongest proposal. Replicas use the following lexicographic ordering to compare proposals:

(1) *Highest PoR:* A proposal whose log contains a PoR from a higher round is always preferred, as it represents a more recent globally reliable certificate.

(2) *Highest PoA:* If two proposals have the same highest PoR round, the one with a higher PoA round is preferred, as it reflects more recent partial progress.

If multiple proposals remain tied in round  $r$ , replicas apply a deterministic common-coin tie-breaker. To support this, every proposal carries the proposer's threshold signature share for round  $r$  (refer to Section 2). Let  $S_r$  be any set of  $f+1$  valid threshold signature shares from distinct proposals in round  $r$ , and aggregating any

**Data Structure** (replica  $p_i$ ):

**Data structures & State**

**Proposal Msg:**  $P \triangleq \langle r, \text{txs}, \text{parent}, \text{enterCert}, \widehat{\text{PoR}}, \widehat{\text{PoA}} \rangle$

**Local State:**  $r_i$  (round),  $\mathcal{L}_i$  (log),  $\widehat{\text{PoR}}_i$ ,  $\widehat{\text{PoA}}_i$ ,  $\text{lock}_i$

**Buffer State:**  $\mathcal{P}_i[r]$  (proposals),  $\text{voteSet}_i$  (votes),  $\text{Voted}_i$  (flag)

**Local Timers:**  $\delta_i$ , *ExchangeTimer*, *ElectionTimer*

**Helper Functions** (replica  $p_i$ ):

```

1: function VALIDENTERCERT(cert, r) do
2:   return cert is well-formed and one of:
3:     (i) cert = genesis and  $r = 1$ , or
4:     (ii) cert is PoR with round(cert) =  $r - 1$ , or
5:     (iii) cert is RC with round(cert) =  $r - 1$ 

6: function SAFETOVOTE(P) do
7:   return P is valid and either
8:     (i) P extends  $\text{lock}_i$ , or
9:     (ii) P carries a PoR from a strictly higher round

10: function SELECTSTRONGEST( $\mathcal{P}$ ) do
11:   return the proposal in  $\mathcal{P}$  with highest priority ▷Section 4.1

12: event ENTERROUND(r, cert,  $t_x$ ) do
13:   if  $r \leq r_i$  or VALIDENTERCERT(cert, r) = False then return
14:   Cancel ExchangeTimer[ $r_i$ ]; Cancel ElectionTimer[ $r_i$ ]
15:    $r_i \leftarrow r$ ;  $\text{Voted}_i[r_i] \leftarrow \text{False}$ 
16:   if  $\mathcal{P}_i[r_i]$  is uninitialized then  $\mathcal{P}_i[r_i] \leftarrow \emptyset$ 
17:   start ExchangeTimer[ $r_i$ ] with timeout  $t_x$ 
18:   parent  $\leftarrow \text{hash}(\text{TAILOFLOG}(\mathcal{L}_i))$ 
19:    $P^* \leftarrow \text{NEWPROPOSAL}(r_i, \text{txs}, \text{parent}, \text{cert}, \widehat{\text{PoR}}_i, \widehat{\text{PoA}}_i)$ 
20:   ▷ $\widehat{\text{PoR}}_i, \widehat{\text{PoA}}_i$  carry the highest known PoR and PoA
21:   Broadcast( $\langle \text{PROPOSE}(P^*) \rangle$ ) ▷self-proposal
22:    $\mathcal{P}_i[r_i] \leftarrow \mathcal{P}_i[r_i] \cup \{P^*\}$ 

22: function UPDATEHICERT(cert) do
23:   if cert is PoR and round(cert) > round( $\widehat{\text{PoR}}_i$ ) then  $\widehat{\text{PoR}}_i \leftarrow \text{cert}$ 
24:   if cert is PoA and round(cert) > round( $\widehat{\text{PoA}}_i$ ) then  $\widehat{\text{PoA}}_i \leftarrow \text{cert}$ 

25: function MAYBEADVANCELOCKANDCOMMIT( $\text{lock}_i$ , cert) do
26:   ▷Two-PoR Rule—lock on the 1st PoR, commit at the 2nd PoR
27:   if cert and its ancestors form the required PoR chain then
28:     update  $\text{lock}_i$  to the latest safely locked proposal
29:     commit older proposals according to the above rule

```

Figure 4: Utility and Helper Functions for CASSANDRA

valid set of  $f+1$  shares yields the same round coin  $C_r$ .

$$C_r = \text{THRESHOLDAGGREGATE}(S_r)$$

$$\text{Score}(\mathcal{P}) = H(C_r \parallel \mathcal{P}.\text{sender}).$$

The proposal with the highest  $\text{Score}(\cdot)$  is designated as the Strongest Proposal of round  $r$ , which we denote as  $SP_r$ . Each replica then broadcasts a VOTE for  $SP_r$ . If a proposal gathers  $Q = 2f+1$  votes, a PoR is formed. If it times out before forming a PoR but gathers  $Q_{\text{weak}} = f+1$  votes, a PoA is formed.

**4.1.1 Locking and Commitment.** Replicas independently interpret their local logs and finalize proposals by applying a deterministic locking and commit rule over PoRs.

*Locking Rule.* Each replica maintains a local variable  $\text{lock}_i$  representing the highest PoR-certified proposal it has observed. A replica votes only for proposals that extend  $\text{lock}_i$ , unless it observes a PoR from a strictly higher round.

*Commitment Rule.* A proposal  $b_r$  is committed once it forms a two-PoR chain. Formally, if  $b_r$  and its immediate successor  $b_{r+1}$  both obtain PoRs (i.e.,  $\text{PoR}_r$  and  $\text{PoR}_{r+1}$ ), and  $b_{r+1}$  extends  $b_r$ , then  $b_r$  and all its ancestors become committed (Figure 6).

---

**Round Logic** (replica  $p_i$ ):

**Initialization**  
 initialize  $lock_i$ ,  $PoR_i$ ,  $PoA_i$  to genesis;  $r_i \leftarrow 0$   
 Call ENTERROUND(1, genesis,  $2\delta_i$ )

```

1: event Received valid (PROPOSE( $P_r$ )) from  $p_j$  do
2:   if  $P_r.\text{round} < r_i$  or VALIDENTERCERT( $P_r.\text{enterCert}$ ,  $P_r.\text{round}$ ) = False
   then return
3:   UPDATEHiCERT( $P_r.PoR$ ); UPDATEHiCERT( $P_r.PoA$ ) ▷Outdated or invalid
4:   MAYBEADVANCELOCKANDCOMMIT( $lock_i$ ,  $P_r.PoR$ )
   ▷Update local safety state before processing this proposal
5:   if  $P_r.\text{round} > r_i$  then
     ▷Fast-forward to an existing higher round
6:   Call PACEMAKER.OBSERVEHIGHERROUND( $P_r$ )
7:   if SAFETOVOTE( $P_r$ ) then
8:      $\mathcal{P}_i[P_r.\text{round}] \leftarrow \mathcal{P}_i[P_r.\text{round}] \cup \{P_r\}$ 

9: event ExchangeTimer[ $r$ ] timeout ▷Exchange phase ends do
10:  if  $r \neq r_i$  or Voted $_i[r]$  then return
11:   $\mathcal{P}_i^{safe}[r] \leftarrow \{P \in \mathcal{P}_i[r] \mid \text{SAFETOVOTE}(P)\}$ 
12:  if  $\mathcal{P}_i^{safe}[r] = \emptyset$  then
13:    EXITROUND( $r$ ,  $\perp$ ); return
14:   $P^{SP} \leftarrow \text{SELECTSTRONGEST}(\mathcal{P}_i^{safe}[r])$ 
15:   $\text{voteSet}_i[r, \text{hash}(P^{SP})] \leftarrow \text{voteSet}_i[r, \text{hash}(P^{SP})] \cup \{p_i\}$ 
16:  Broadcast((VOTE( $r$ ,  $\text{hash}(P^{SP})$ )))
17:  Voted $_i[r] \leftarrow \text{True}$ 
18:  start ElectionTimer[ $r$ ] with timeout  $\delta_i$ 

19: event Received valid (VOTE( $r$ ,  $h$ )) from  $p_j$  ▷Collect votes do
20:  if  $r \neq r_i$  then return
21:   $\text{voteSet}_i[r, h] \leftarrow \text{voteSet}_i[r, h] \cup \{p_j\}$ 
22:   $c \leftarrow |\text{voteSet}_i[r, h]|$ 
23:  if  $c \geq Q$  then ▷strong quorum  $Q = 2f+1$ 
24:    ( $PoR$ ,  $h$ )  $\leftarrow$  BUILDERT( $r$ ,  $h$ ,  $c$ ,  $PoR$ )
25:    append ( $PoR$ ,  $h$ ) to  $\mathcal{L}_i$ 
26:    UPDATEHiCERT(( $PoR$ ,  $h$ ))
27:    MAYBEADVANCELOCKANDCOMMIT( $lock_i$ ,  $PoR_i$ )
28:    EXITROUND( $r$ , ( $PoR$ ,  $h$ ))

29: event ElectionTimer[ $r$ ] timeout ▷Election phase ends do
30:  if  $r \neq r_i$  then return
31:   $h^* \leftarrow \arg \max_h |\text{voteSet}_i[r, h]|$ 
32:   $c^* \leftarrow |\text{voteSet}_i[r, h^*]|$ 
33:  if  $c^* \geq Q_{weak}$  then ▷weak quorum  $Q_{weak} = f+1$ 
34:    ( $PoA$ ,  $h^*$ )  $\leftarrow$  BUILDERT( $r$ ,  $h^*$ ,  $c^*$ ,  $PoA$ )
35:    append ( $PoA$ ,  $h^*$ ) to  $\mathcal{L}_i$ 
36:    UPDATEHiCERT(( $PoA$ ,  $h^*$ ))
   ▷PoA preserves progress, but does not justify next-round entry
37:  EXITROUND( $r$ ,  $\perp$ )

38: event EXITROUND( $r$ , cert) do
39:  if  $r \neq r_i$  then return
40:  Cancel ExchangeTimer[ $r$ ]; Cancel ElectionTimer[ $r$ ]
41:  Call PACEMAKER.COMPLETEROUND( $r$ , cert)

```

---

Figure 5: Pseudocode of Round Logic in CASSANDRA

**Safety Intuition.** Each PoR is formed by a strong quorum of size  $2f+1$ . Any two such quorums intersect in at least  $f+1$  replicas, including at least one honest replica. Once  $PoR_r$  and  $PoR_{r+1}$  are formed for a chain extending  $b_r$ , at least  $f+1$  honest replicas participating in  $PoR_{r+1}$  become locked on the proposal containing  $b_r$ . Due to quorum intersection and the single-vote rule, no conflicting proposal can subsequently gather a strong quorum in round  $r+1$  or beyond. Hence, two conflicting proposals cannot both satisfy the two-PoR commit condition, ensuring the safety of consensus.

## 4.2 Round Synchronization

To form either a PoR or a PoA, replicas must remain in the same round long enough to exchange proposals and votes. Classical pacemakers often place this synchronization logic directly on the

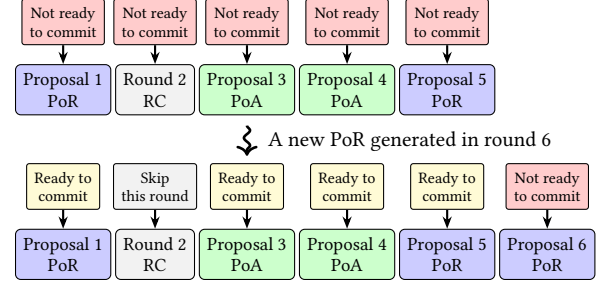


Figure 6: An Example of Two-PoR Commit Rule

critical consensus path and require a strong quorum of replicas for synchronization. Under network partitions, such designs stall.

CASSANDRA instead deploys a *decoupled pacemaker*: round advancement remains certificate-driven on the critical path, while timeouts are calibrated off the critical path through a background synchronization service. This design simultaneously supports: (i) partial progress within weak connected components of at least  $f+1$  replicas, and (ii) fast catch-up and convergence once  $2f+1$  replicas become connected.

**4.2.1 Round Advancement Conditions.** After the election and certification phase, replicas participate in the round advancement phase. A replica in round  $r$  advances to round  $r+1$  in exactly one of the following two cases:

- **PoR-based.** If a proposal in round  $r$  gathers a strong quorum of votes and forms a  $PoR_r$ , replicas advance to round  $r+1$ .
- **RC-based advancement.** If no proposal forms a  $PoR_r$ , before the election timer expires, the replica broadcasts a WISHPROPOSE( $r$ ) message. Once it collects  $f+1$  such messages, it forms a *Round Certificate* ( $RC_r$ ) and advances to round  $r+1$ .

Crucially, these rules control logical round advancement in the consensus path: replicas continue progressing through proposal exchange, voting, and PoR/RC formation without waiting for a separate timeout synchronization procedure to complete.

**4.2.2 Quick Catch-Up and Fast Convergence.** Once a replica forms a PoR or RC certificate, it broadcasts the certificate so that any lagging replica can immediately jump to the *justified* round. Concretely, if a replica is currently in round  $r'$  and receives a valid PoR or RC formed in round  $r > r'$ , it advances to round  $r+1$ .

Since round numbers increase monotonically and replicas only accept certificates for strictly higher rounds, certificate dissemination induces convergence toward the highest justified round. If a lagging replica receives  $PoR_r$ , it learns that round  $r$  has successfully produced a strong certificate and advances accordingly. If it receives  $RC_r$ , it learns that an honest replica in round  $r$  could not form a PoR and synchronizes by advancing to round  $r+1$ .

During synchronous periods after GST, such certificates propagate within  $\Delta$ , bounding round divergence among correct replicas by at most one message delay. This also explains why the *Proposal Exchange* phase spans  $2\delta_i$ : one  $\delta_i$  for round synchronization and one  $\delta_i$  for proposal dissemination. In particular, once a strong quorum becomes connected again, certificate dissemination rapidly pulls lagging replicas toward the highest justified round, without requiring an explicit synchronization barrier on the critical path.

---

**Pacemaker<sub>i</sub>** (Round Advancement & Timeout Calibration) :

**Local state:**  
 $r_i, K$  ▷ current round and timeout calibration interval  
 $\delta_i, v_i$  ▷ local timeout baseline and current sync view  
 $Calibrating_i$  ▷ whether a calibration attempt is in progress  
 $ReadySet[v], CertSet[v]$  ▷ valid <SYNC> messages

---

```

1: function COMPLETEROUND( $r, cert$ ) do
2:   if  $r \bmod K = 0$  then ▷ Reach the SYNC TIMEOUT boundary.
3:     Call PACEMAKER.SYNCTIMEOUT
4:   if  $cert$  is PoR then
5:     Call ENTERROUND( $r+1, cert, 2\delta_i$ )
6:     ▷ A PoR directly justifies entry into the next round
7:   else Broadcast(WISHNEWROUND( $r$ ))
8:     ▷ If no PoR, request for an RC for round advancement

7: event Received  $f+1$  (WISHNEWROUND( $r$ )) messages do
8:    $\langle RC_r \rangle \leftarrow \text{AGGREGATESIG}(f+1 \text{ (WISHNEWROUND}(r))$ )
9:   Broadcast( $\langle RC_r \rangle$ )

10: event Received  $\langle RC_r \rangle$  do
11:   Call ENTERROUND( $r+1, RC_r, 2\delta_i$ )
12:   ▷ An  $RC_r$  safely coordinates entry into round  $r+1$ 

12: function OBSERVEHIGHERROUND( $P$ ) do
13:   if  $P.r \leq r_i$  or VALIDENTERCERT( $P, enterCert, P.r$ ) = False then return
14:   ▷ The enterCert certifies the round entry
15:   ▷ Use a shortened timeout since round  $r'$  is already in progress
16:   Call ENTERROUND( $P.r, P.enterCert, \delta_i$ )

15: event SYNCTIMEOUT do
16:   if  $Calibrating_i = \text{False}$  then
17:      $v_i \leftarrow v_i + 1; Calibrating_i \leftarrow \text{True}$ 
18:     Broadcast( $\langle \text{SYNC-READY}, v_i \rangle_i$ )

19: event Received valid  $\langle \text{SYNC-READY}, v \rangle$  from  $p_j$  do
20:    $ReadySet[v][j] \leftarrow \langle \text{SYNC-READY}, v \rangle_j$ 
21:    $V^{hi} \leftarrow \{v' \mid v' > v_i \text{ and } |ReadySet[v']| \geq f+1\}$ 
22:   if  $V^{hi} \neq \emptyset$  then
23:      $v_i \leftarrow \min V^{hi}; Calibrating_i \leftarrow \text{True}$ 
24:     Broadcast( $\langle \text{SYNC-READY}, v_i \rangle_i$ )
25:   if  $CertSet_i[v][i] = \perp$  and  $|ReadySet[v]| \geq 2f+1$  then
26:      $CertSet_i[v][i] \leftarrow \langle \text{SYNC-CERT}, v \rangle_i$ 
27:     Broadcast( $\langle \text{SYNC-CERT}, v \rangle_i$ )
28:     start SyncTimer[ $v$ ] with timeout  $\delta_i$ 

29: event Received valid  $\langle \text{SYNC-CERT}, v \rangle$  from  $p_j$  do
30:    $CertSet[v][j] \leftarrow \langle \text{SYNC-CERT}, v \rangle_j$ 
31:   if  $CertSet[v][i] \neq \perp$  and  $|CertSet[v]| > f+1$  then
32:      $Calibrating_i \leftarrow \text{False}; \text{Cancel SyncTimer}[v]$ 
33:     ▷  $\delta_i$  is sufficient; decrease if success was fast (Section 4.2.3)

33: event SyncTimer[ $v$ ] timeout do
34:   if  $|CertSet[v]| \geq f$  then
35:      $\delta_i \leftarrow 2\delta_i; v_i \leftarrow v_i + 1$ 
36:     Broadcast( $\langle \text{SYNC-READY}, v_i \rangle_i$ )

```

---

Figure 7: Pseudocode of the Pacemaker

**4.2.3 Background Timeout Calibration.** CASSANDRA operates under a partially synchronous network model, in which message delays are bounded by some  $\Delta$  that holds only during stabilized periods (after some GST) and whose value is not known a priori. Because the actual delay can differ across stable periods—particularly after long partitions or asymmetric delays—replicas’ local timeouts may become misaligned. To address this, existing pacemakers perform timeout calibration, but they do so on the critical path.

To address this, CASSANDRA runs a periodic background timeout-calibration service, denoted SyncTimeout, which adjusts each replica’s local timeout value ( $\delta_i$ ). This service is *off the critical path*: it never blocks the chain construction process. Replicas initiate SyncTimeout

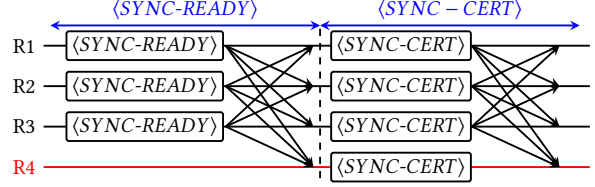


Figure 8: Timeout Calibration

periodically, e.g., once every  $K$  rounds ( $K$  is a system parameter), provided that no prior calibration attempt is still in progress.

Each calibration attempt is identified by a synchronization view number  $v$ . The view number is used to distinguish retries and prevent replay of stale synchronization messages. If a calibration attempt fails, replicas keep retrying in the background using higher synchronization views, while the main protocol remains unchanged and continues running.

**Phase 1: <SYNC-READY>.** After every  $K$ -th round, if replica  $p_i$  is not undergoing a timeout-calibration, it initiates a calibration attempt in synchronization view  $v$  by broadcasting  $\langle \text{SYNC-READY}, v \rangle_i$ . Replica  $p_i$  keeps periodically retransmitting this message until it receives either  $2f+1$  such messages for view  $v$  or  $f+1$  such messages for higher views  $v'$ . In the latter case, replica  $p_i$  abandons view  $v$  and joins the smallest view  $v'$  among these  $f+1$  messages by broadcasting  $\langle \text{SYNC-READY}, v' \rangle_i$ .

**Phase 2: <SYNC-CERT>.** Once a replica  $p_i$  collects  $2f+1$  valid SYNC-READY messages for its synchronization view  $v$ , it creates and broadcasts a  $\langle \text{SYNC-CERT}, v \rangle_i$  message. After that,  $p_i$  waits on a timer of length  $\delta_i$  for SYNC-CERT messages from other replicas.

**Success Condition.** The calibration attempt in view  $v$  succeeds at replica  $p_i$  if:  $p_i$  formed a  $\langle \text{SYNC-CERT}, v \rangle_i$ , and within period  $\delta_i$ , it received  $\langle \text{SYNC-CERT}, v \rangle$  from at least  $f+1$  distinct replicas (i.e., the total number of certificates, including its own, exceeds  $f+1$ ).

This condition ensures that replica  $p_i$ ’s current timeout  $\delta_i$  is long enough to allow at least one honest replica to generate and send back a certificate under the current network condition. In this case, replica  $p_i$  keeps its current timeout baseline unchanged.

**Failure handling.** We distinguish two failure cases:

(1) **No local certificate formed.** If replica  $p_i$  cannot collect  $2f+1$  valid SYNC-READY messages for view  $v$ , it leaves  $\delta_i$  unchanged and keeps retransmitting  $\langle \text{SYNC-READY}, v \rangle_i$ .

(2) **Insufficient remote certificates received.** If replica  $p_i$  succeeds in forming  $\langle \text{SYNC-CERT}, v \rangle_i$  but does not receive this certificate from at least  $f+1$  distinct replicas within time  $\delta_i$ , the attempt is treated as a failure. In this case, replica  $p_i$  doubles its local timeout period, i.e.,  $\delta_i \leftarrow 2\delta_i$ , advances to the next synchronization view  $v \leftarrow v+1$ , and restarts the procedure by broadcasting a new  $\langle \text{SYNC-READY}, v \rangle_i$ .

**Timeout Decrease.** Classical BFT protocols such as PBFT [10] double the view-change timeout whenever a view change fails to make progress, but never decrease it. As a result, a single transient period of high delay can leave rounds unnecessarily long well after the delay has subsided. CASSANDRA optionally permits decreasing  $\delta_i$  upon a sufficiently fast calibration success. Concretely, if a calibration attempt succeeds and the elapsed time since replica  $p_i$  formed its local  $\langle \text{SYNC-CERT}, v \rangle_i$  is less than  $\delta_i/\alpha$  for a pre-set

---

**Pacemaker (Timeout Calibration)** (replica  $p_i$ ):

**Local State**

- $\delta_i, v_i$  ▷ local timeout baseline and current sync view
- $\text{Calibrating}_i$  ▷ whether a calibration attempt is in progress
- $\text{ReadySet}[v]$  ▷ valid {SYNC-READY,  $v$ } messages
- $\text{CertSet}[v]$  ▷ valid {SYNC-CERT,  $v$ } messages

```

1: event SyncTimeout do
2:   if Calibratingi = False then
3:     vi ← vi + 1; Calibratingi ← True
4:     Broadcast((SYNC-READY, vi))i

5: event Received valid {SYNC-READY, v} from pj do
6:   ReadySet[v][j] ← {SYNC-READY, v}j
7:   Vhi ← {v' | v' > vi and |ReadySet[v']| ≥ f + 1}
8:   if Vhi ≠ ∅ then
9:     vi ← min Vhi; Calibratingi ← True
10:    Broadcast((SYNC-READY, vi))i
11:   if CertSet[v][i] = ⊥ and |ReadySet[v]| ≥ 2f + 1 then
12:     CertSet[v][i] ← {SYNC-CERT, v}i
13:     Broadcast((SYNC-CERT, v))i
14:     start SyncTimer[v] with timeout δi

15: event Received valid {SYNC-CERT, v} from pj do
16:   CertSet[v][j] ← {SYNC-CERT, v}j
17:   if CertSet[v][i] ≠ ⊥ and |CertSet[v]| > f + 1 then
18:     Calibratingi ← False; Cancel SyncTimer[v]
19:     ▷ δi is sufficient; decrease if success was fast (Section 4.2.3)

19: event SyncTimer[v] timeout do
20:   if |CertSet[v]| ≤ f then
21:     δi ← 2δi; vi ← vi + 1
22:     Broadcast((SYNC-READY, vi))i

```

---

Figure 9: Pseudocode of Background Timeout Calibration

threshold  $\alpha > 1$  (e.g.,  $\alpha = 4$ ),  $p_i$  halves its local timeout baseline:  $\delta_i \leftarrow \max(\delta_i/2, \delta_{\min})$ , where  $\delta_{\min}$  is a pre-configured lower bound. The subsequent calibration attempt then verifies whether the reduced timeout is still sufficient; if not, it is doubled back. This yields a multiplicative-increase/multiplicative-decrease pattern analogous to TCP congestion control [27], adapted to the BFT setting where safety is never at risk because  $\delta_i$  affects only round duration, not the correctness of voting or certification.

### 4.3 Merge after Network Recovery

A PoA-backed branch guarantees data availability, provides ordering evidence, and reduces wasted work after recovery. However, it is provisional and does not imply final commitment. During a network partition, different connected components may therefore extend the same PoR along divergent PoA chains. The same reasoning applies to asymmetric partitions in which a Byzantine replica selectively forwards proposals across components: forwarding may create additional provisional PoA branches, which introduces additional computation costs linear in the number of PoAs for verification, but does not force a pairwise merge of all branches or roll back any committed state.

When network connectivity is restored, replicas resume exchanging proposals in the *proposal exchange* phase. Each proposal carries its sender's highest known PoR and PoA, summarizing the strongest certificates that replica has observed. During the election phase, replicas then apply the deterministic proposal-priority rule: proposals anchored at a higher-round PoR are always preferred, and ties are broken by the highest-round PoA and then the threshold coin.

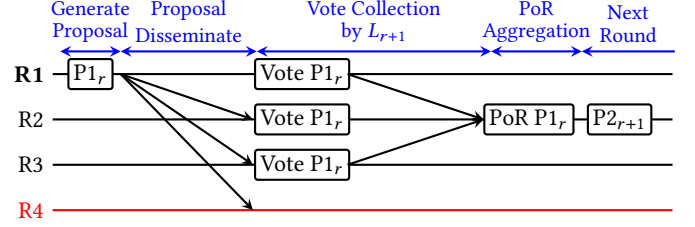


Figure 10: Fast Path Workflow in Good Cases

Consequently, once proposals from previously disconnected components become visible across the network, correct replicas deterministically converge on the same strongest branch and vote for it. This allows a new PoR to form on that branch, re-establishing a single, globally agreed PoR. The remaining PoA branches—those not extended by a subsequent PoR—stop growing and are eventually superseded. Because PoA formation guarantees data availability, the proposals on these branches remain retrievable; however, only the branch incorporated into a subsequent PoR can be finalized.

Merging in CASSANDRA is therefore implicit and protocol-driven: no separate reconciliation procedure is required, as the normal proposal-selection and certification process automatically converges the system back to a single chain once connectivity is restored.

## 5 CASSANDRA: DUAL-PATH DESIGN

Although CASSANDRA achieves partial progress under network partitions, it relies on all-to-all proposal exchange and conservative timeouts; as a result, it incurs *quadratic communication complexity* and forgoes *optimistic responsiveness* [52], preventing replicas from proceeding at network speed even when the network is stable. In this section, we present an optimized version of CASSANDRA based on a *dual-path design*. In the absence of leader or network failures, the system runs an optimistic *fast path* that assumes a *designated* leader and lets replicas reach consensus with linear communication. If the *fast path* fails, replicas fall back to the *base path* (§4).

### 5.1 Linear-Complexity Fast Path

CASSANDRA's fast-path assumes a dedicated leader for each round; let  $L_r$  be the leader for round  $r$  (leaders are selected in a round-robin manner). The fast-path undergoes the following steps (Fig. 10).

- **Fast-path proposal.** On entering round  $r$  (i.e., on receiving a  $PoR_{r-1}$  or an  $RC_{r-1}$ ), the leader  $L_r$  broadcasts a proposal  $b_r$ . This proposal extends the highest known branch and carries the sender's latest  $\widehat{PoR}$  and  $\widehat{PoA}$  certificates (similar to the base path).

- **Linear vote collection.** Upon receiving  $b_r$ , each replica verifies that it is valid and lock-compatible (cf. Section 4.1.1). If this is the case, the replica sends its vote share to a designated collector, e.g., the next-round leader  $L_{r+1}$ .

- **Pipelined certificate formation.** Once  $L_{r+1}$  collects a strong quorum of vote shares for  $b_r$ , it aggregates them into a  $PoR_r$ . This  $PoR_r$  then serves as the justification to enter round  $r+1$ , allowing  $L_{r+1}$  to immediately propose the next block.

In summary, the fast path modifies only the *message-collection pattern* used to form a PoR; it does not alter the safety rule, the locking rule, or the certificate semantics. As a result, under stable conditions, CASSANDRA substantially reduces its communication complexity, from quadratic to linear.

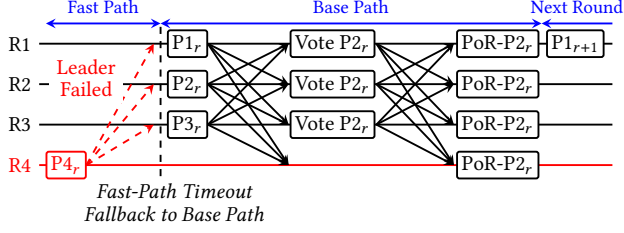


Figure 11: Fallback from Fast Path to Base Path

## 5.2 Fallback to the Base Path

Each replica maintains a dedicated *Fast-Path Timer* to detect failures. For example, if a replica does not receive a valid fast-path proposal from  $L_r$  in time, or if the designated collector fails to gather a strong quorum of vote shares before the timer expires, the replica abandons the fast path and falls back to the *base path* (Fig. 11).

The base Path resumes within the same round and follows the steps described in Section 4.1. Concretely, each replica broadcasts its own round- $r$  proposal and participates in the *proposal exchange* and *election and certification* phases. These proposals include locally known highest *PoR* and *PoA* certificates, so safety and proposal priority remain unchanged.

On entering the next round, replicas again attempt the fast path. The transition between the fast and base paths is therefore seamless: the fast path is a performance optimization layered on the base protocol, not a separate consensus protocol.

## 5.3 Dissemination Layer in CASSANDRA

Prior work such as Narwhal [14] and AutoBahn [19] introduce the notion of a *dissemination layer* that decouples data dissemination from ordering. Concretely, the dissemination layer continuously broadcasts batches of transactions and certifies their availability across replicas, while the ordering layer concurrently proposes references to the already-disseminated data. This decoupling is complementary to CASSANDRA's design and can be integrated with modest changes.

## 5.4 Speculative Execution over PoA

A PoA certificate in CASSANDRA does not guarantee final commitment, but it does certify that a proposal has made recoverable partial progress. To avoid leaving this certified work idle during a partition, replicas may speculatively execute PoA-backed proposals before they are committed via a PoR. The execution result can be returned to the client as speculative output, though it does not constitute a final commit. After recovery, if the branch becomes the strongest branch, its speculative execution is reused on the committed path; otherwise, the speculative state is discarded, and its transactions must be re-proposed through the normal path. Speculative execution thus provides a meaningful way to measure partial progress.

## 6 PROOF SKETCH

This section summarizes the main arguments behind CASSANDRA's safety, liveness under partial synchrony, and partial liveness. Full proofs are deferred to Appendix A [50].

**Theorem 1 (Safety).** *No two correct replicas commit (finalize) conflicting proposals.*

**PROOF SKETCH.** The argument follows from three ingredients. First, any two strong quorums of size  $2f+1$  intersect in at least  $f+1$  replicas, and therefore share at least one correct replica. Second, a correct replica casts at most one vote per round. Hence, two conflicting proposals cannot both obtain valid PoR certificates in the same round.

Across rounds, safety is preserved by the locking rule. Once a correct replica observes a PoR for a proposal, it locks that proposal (or a later descendant on the same branch) and will not vote for a conflicting one unless it observes a strictly higher-round PoR on the competing proposal's certified ancestry. Therefore, once a proposal becomes commit-ready under the Two-PoR rule, the second PoR contains at least  $f+1$  correct voters that are locked on that branch. By quorum intersection, any later strong quorum for a conflicting branch must intersect this locked set in at least one correct replica, which prevents the conflicting branch from gathering the certified chain needed for commitment.  $\square$

## 6.1 Liveness

**Theorem 2 (Liveness (with probability 1)).** *During any sufficiently long stabilized synchronous period, correct replicas commit new proposals continuously; over an unbounded sequence of such stabilized periods, commits occur infinitely often with probability 1.*

**PROOF SKETCH.** During a sufficiently long stabilized synchronous period, the background timeout-calibration mechanism eventually makes every correct replica's local timeout baseline  $\delta_i$  sufficiently large relative to the actual network delay bound. Once local timeouts are sufficient, PoRs and RCs are delivered within the underlying synchronous delay bound. Since every valid higher-round proposal carries one of these certificates as its *enterCert*, certificate dissemination together with proposal-driven fast catch-up eventually re-aligns correct replicas into recurring *good rounds*, in which all correct replicas enter the same round early enough to complete proposal exchange and vote collection.

In each good round, all correct replicas observe the same set of honest proposals. If there is a unique strongest proposal, they all vote for it directly. Otherwise, ties are broken by the common coin; with probability at least  $\frac{2f+1}{n}$ , the selected top-scoring proposer is honest, in which case all correct replicas converge on the same proposal and form a valid PoR. Since good rounds recur and each one succeeds with non-zero probability, PoRs occur infinitely often almost surely. Whenever two consecutive rounds produce extending PoRs, the Two-PoR rule commits the earlier proposal in the chain.  $\square$

## 6.2 Partial Liveness

**Theorem 3 (Partial Liveness).** *CASSANDRA satisfies the **Partial Liveness** property. Specifically, during network partitions, any connected component that contains at least  $f+1$  correct replicas and whose internal message delays are bounded by  $\Delta$ , and whose correct replicas use local timeout baselines satisfying  $\delta_i \geq \Delta$ , continues to advance rounds locally and produce PoA-certified partial progress. After network recovery, this accumulated partial progress remains recoverable and can be incorporated into subsequent PoR-backed commitment.*

**PROOF SKETCH.** Consider any connected component containing at least  $f+1$  correct replicas and timely internal communication. Within such a component, correct replicas can still exchange proposals within one local round and apply the same priority rule to identify a common strongest candidate. As a result, the component can assemble a weak quorum and form a PoA, thereby preserving certified partial progress even without a global strong quorum.

If no PoR is formed in that round, correct replicas eventually time out and broadcast WISHPROPOSE; since the component contains at least  $f+1$  correct replicas, they can form an RC for the next round and continue advancing locally. Thus, under partition, CASSANDRA preserves both round progression and recoverable partial progress.

A PoA does not imply commitment, but it does imply data availability: at least one correct replica stores the certified proposal. Once global connectivity is restored, a strong quorum becomes reachable again, and replicas resume forming PoRs on the strongest recoverable branch. Through the catch-up and back-fill mechanisms, previously accumulated PoA-backed progress can be retrieved, extended, and incorporated into the committed prefix.  $\square$

## 7 EVALUATION

To validate our partial-progress claims, we implement CASSANDRA and evaluate it against several state-of-the-art BFT protocols: Tusk [14], AutoBahn [19], PBFT [9], HotStuff [52], SpotLess [29], and RCC [22]. Our evaluation targets two key metrics—**throughput** and **latency**—and aims to address the following research questions:

- **Q1:** How does CASSANDRA scale under stable network conditions compared with existing BFT protocols? (Section 7.2)
- **Q2:** How does CASSANDRA behave under network partitions, from  $f$  partitioned replicas to balanced partitions? (Section 7.3)
- **Q3:** How robust is CASSANDRA under Byzantine behaviors such as intentional delay and tail-forking attacks? (Section 7.4)
- **Q4:** What do the ablation study and latency breakdown reveal about CASSANDRA's performance? (Section 7.5)
- **Q5:** How does CASSANDRA perform under geo-distributed regional partitions? (Section 7.6)

### 7.1 Implementation and Evaluation Setup

**Implementation.** We implement CASSANDRA in C++ [1] as a core consensus component within the open-source Apache ResilientDB platform [5, 23], a scalable blockchain infrastructure. The prototype reuses ResilientDB's networking, batching, execution, monitoring, and key-management pipeline. Messages are authenticated through ResilientDB's Crypto++ verifier, and our experiments use Ed25519 replica keys generated by ResilientDB's key-generation tool. PoA, PoR, and RC certificates are formed by collecting authenticated replica votes and checking corresponding thresholds. Unless stated otherwise, CASSANDRA enables the fast path, the dissemination layer, and speculative execution. Speculative execution lets replicas execute a PoA-backed proposal before final PoR commitment. This avoids long execution queues and reduces executor idleness during partitions, which in turn speeds up recovery once the network is restored. For experiments involving network partitions, we report *speculative throughput*, which counts PoA-backed proposals that have been locally ordered and executed but not yet finally committed.

**Evaluation Setup.** We deploy c5.4xlarge instances (16 vCPUs, 32 GB memory) on AWS in Northern Virginia. For geo-distributed experiments, we use four regions: Northern Virginia, São Paulo, Frankfurt, and Singapore. Each replica is assigned a dedicated client so that all proposers propose at the same rate, and each client issues transactions in an open loop at a rate sufficient to saturate its replica. We also *batch transactions* for processing. For benchmarking, we use ResilientDB's key-value service, in which each transaction is a randomly generated key-value insertion of a fixed size (50 bytes). All reported numbers are averaged over three runs.

### 7.2 Performance Under Normal Conditions

First, we study the scalability of CASSANDRA under stable network conditions (no failures). Figure 12(a) varies the number of replicas from 16 to 104. All protocols exhibit a similar trend: as the number of replicas grows, throughput decreases and latency increases, because more participants must reach agreement. All protocols except PBFT and HotStuff sustain high throughput, as they are not constrained by a single-leader design. PBFT achieves higher throughput than HotStuff because its replicas can process transactions out of order, whereas HotStuff's rotating-leader design prohibits this. CASSANDRA has a deterministic proposal-priority rule and dissemination, while Tusk, AutoBahn, RCC, and SpotLess employ similar multi-proposer or decoupled data paths. CASSANDRA's partial-progress mechanisms, however, marginally increase latency relative to the lowest-latency baselines. At 104 replicas, CASSANDRA's throughput is within 1.5% of AutoBahn's.

### 7.3 Performance under Network Partitions

Next, we study the impact of network partitions on different protocols by increasing the number of partitioned replicas.

**7.3.1  $0 \rightarrow f \rightarrow 0$  Partition.** First, we ensure that only  $f$  replicas are partitioned (from 24s to 44s), i.e., a strongly connected component of  $2f+1$  replicas still exists (Figure 12(b)). All protocols except PBFT experience a drop in throughput (and a rise in latency) during this partition; PBFT remains relatively stable because its leader lies within the  $2f+1$  connected component. HotStuff, Tusk, and AutoBahn degrade due to their rotating-leader designs. Tusk's wave-based consensus periodically completes a large batch of requests whenever the leader is not partitioned away, while AutoBahn's dissemination layer helps it sustain higher throughput after a brief period of zero throughput caused by its HotStuff-style ordering layer. RCC and SpotLess both run multi-leader protocols: after an initial drop, RCC retains two-thirds of its steady-state performance once it excludes the  $f$  partitioned replicas, whereas SpotLess's multiple rotational-leader instances make it behave similarly to AutoBahn, since some views always have partitioned leaders. CASSANDRA, in contrast, continues forming PoR certificates and finalizing transactions. It does incur a 33.7% throughput drop relative to stable settings, due to the smaller set of connected replicas and the switching between fast and base paths.

**7.3.2  $f \rightarrow f+1 \rightarrow f$  Partition.** Next, we assume all protocols run in a state where  $f$  replicas are always partitioned, and then (from 24s to 32s) one additional replica is partitioned, bringing the total to  $f+1$ , so that the largest connected component contains only  $2f$

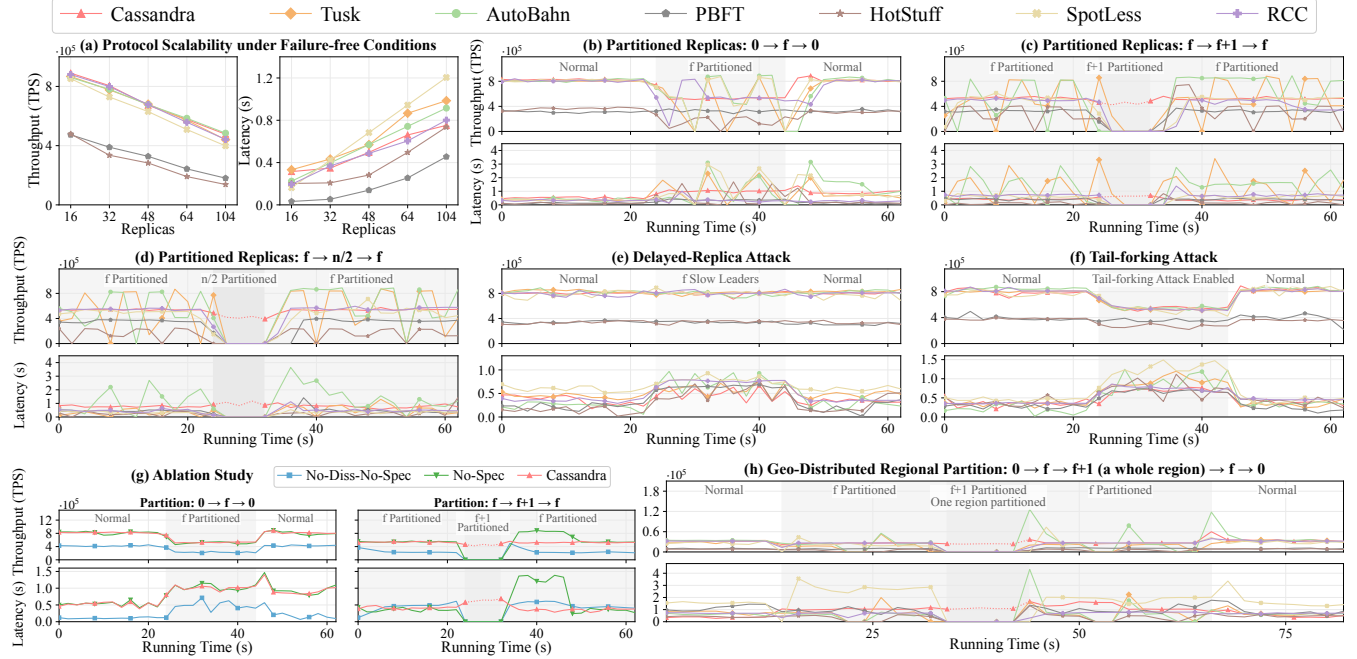


Figure 12: Evaluation summary across scalability and partition scenarios.

replicas (Figure 12(c)). All protocols except CASSANDRA drop to zero throughput once more than  $f$  replicas are partitioned, since they require a strong quorum to run consensus. CASSANDRA, in contrast, observes only a 15% drop in throughput (and a 52% increase in latency), because PoAs require just  $f+1$  votes and thus continue to drive partial progress through speculative execution. Once the system returns to an  $f$ -replica partition, the strong quorum becomes reachable again and all protocols resume committing blocks. AutoBahn shows the largest throughput burst, as its dissemination layer keeps operating as long as  $f+1$  replicas remain connected; however, because those proposals cannot be ordered during the partition, it also exhibits high latency.

**7.3.3  $f \rightarrow \lfloor n/2 \rfloor \rightarrow f$  Partition.** Next, we consider a scenario in which roughly 50% of all replicas are partitioned (from 24s to 32s), leaving a single connected set of  $\lfloor n/2 \rfloor$  replicas (Figure 12(d)). As in the previous experiment, all protocols except CASSANDRA drop to zero throughput, since they stop processing transactions altogether. In CASSANDRA, the connected replicas can still generate PoAs and speculatively execute transactions, though throughput falls by 21% due to the smaller connected set. Once the system is restored to an  $f$ -replica partition, all protocols behave as described above. For CASSANDRA, however, the recovery burden is greater than in §7.3.2, as far more replicas must be reconciled.

## 7.4 Performance under Byzantine Behavior

Next, we study the impact of Byzantine attacks on the different protocols. First, in Figure 12(e), we consider a leader delay attack in which  $f$  Byzantine replicas delay proposals and messages by 50 ms. All protocols show only a marginal drop in throughput and return to steady state quickly, but they do suffer a large latency spike. CASSANDRA incurs only a 40% latency increase during the

delay interval, whereas HotStuff increases by nearly 300%. This is because CASSANDRA can switch between the fast and base paths when the leader is delayed.

Next, we evaluate the impact of a tail-forking attack (Figure 12(f)). In a tail-forking attack,  $f$  faulty replicas are strategically placed across rounds and leader positions to create short forks or unfinished tails. As a result, all protocols except PBFT experience a sustained throughput drop, as proposals are dropped before they can commit—under leader rotation, the next leader may itself be faulty. PBFT’s throughput is unaffected, since we do not designate its leader as faulty. CASSANDRA’s latency, however, is less volatile than that of the other protocols: whenever the leader is faulty, replicas fall back to the base path and select the strongest proposal. This raises latency but keeps it predictable. In contrast, the other protocols rely on their ordering layer to resolve these tails.

## 7.5 Ablation Study and Latency Breakdown

In Figure 12(g), we isolate the effect of CASSANDRA’s various optimizations under network partitions: (i) *No-Diss-No-Spec* disables both the dissemination layer and speculative execution; and (ii) *No-Spec* enables dissemination but disables speculative execution.

In the  $0 \rightarrow f \rightarrow 0$  case, a strongly connected component remains reachable, so the real impact comes from enabling or disabling the dissemination layer. Enabling dissemination improves partition-period throughput by 2.27 $\times$ , because unselected proposals remain available and can be reused rather than discarded as wasted work.

In the  $f \rightarrow f+1 \rightarrow f$  case, the middle interval lacks a strong quorum, so this experiment isolates the benefit of speculative execution. Without it, the *No-Diss-No-Spec* and *No-Spec* variants yield zero speculative throughput during the interval in which  $f+1$  replicas are partitioned (24s to 32s). The dissemination layer’s benefits

appear only once a strongly connected component exists, since by then transactions are already available for ordering.

**Latency Breakdown.** Next, we dissect CASSANDRA's latency to understand where it actually spends time. Roughly 70% of CASSANDRA's latency goes to forming a PoR certificate—which requires collecting votes and disseminating the certificate—and to committing, which entails forming two consecutive PoRs. Proposal dissemination and execution each contribute 10%, strongest-proposal election 6%, cryptography only 1%, and other overheads 3%. Thus, cryptography and proposal selection are not the bottleneck; latency is dominated by the commit path and by communication and certificate coordination.

## 7.6 Geo-Distributed Regional Partition

Finally, in Figure 12(h), we study the impact of deploying replicas in a geo-distributed setting: we deploy  $n = 31$  replicas across four regions, with regional group sizes of 8/8/8/7. The experiment follows the path: normal connectivity  $\rightarrow f$  partitioned replicas  $\rightarrow f+1$  partitioned replicas  $\rightarrow f$  partitioned replicas  $\rightarrow$  recovery. We distribute the  $f$  partitioned replicas across the four regions as 1/1/1/6, and the  $f+1$  partitioned replicas as 1/1/1/7, which renders one full region unavailable.

WAN latency, bandwidth heterogeneity, and cross-region quorum communication reduce throughput and increase latency for all protocols, but the qualitative behavior matches the partition results above. During the  $f$ -partitioned phase, a quorum of  $2f+1$  replicas remains available, so all protocols continue committing proposals, albeit with a decrease similar to that in §7.3.1. During the  $f+1$ -partitioned phase, all protocols except CASSANDRA are unable to commit any blocks, whereas CASSANDRA continues to speculatively execute blocks. After reconnection, cross-region quorum paths are restored and all protocols return to their steady-state performance.

## 8 RELATED WORK

The design of CASSANDRA builds on several lines of BFT consensus research. We summarize the closest work below and defer additional discussion to the extended version [50].

**Network Partition.** Network partitions expose the tension between consistency and availability: traditional BFT protocols preserve safety but stall when a strong quorum ( $2f+1$ ) is unavailable. *Partitionable Blockchain Consensus* [26] formalizes this problem and uses external partition detectors to coordinate progress. Raptr [49] finalizes only a common prefix and reconciles divergent suffixes after recovery, while AutoBahn [19] can keep data dissemination active under disruptions. CASSANDRA differs by making partial progress in the ordering path itself: PoA certificates capture recoverable weak-quorum ordering without relying on partition detectors.

**View Synchronization.** View synchronization keeps replicas aligned on rounds and leaders, from PBFT's timeout/backoff mechanism [9] to HotStuff's modular pacemaker [52]. Recent pacemakers reduce communication overhead and improve responsiveness [35–37]. However, these mechanisms still depend on a strong quorum of view-change or new-view evidence, so partitions without  $2f+1$  connected replicas block global round advancement. CASSANDRA keeps

finality tied to strong quorums, but its pacemaker can advance recoverable partial progress using  $f+1$  evidence.

**Dual-Path BFT.** Dual-path protocols such as Zyzzyva, FaB, and SBFT [20, 34, 42] use an optimistic fast path for low latency and a fallback path for faults. These designs are effective in stable networks, but the common path and recovery path remain largely leader-driven. DAG-based Bullshark [46] also uses a leader in the synchronous path and a coin-assisted fallback for convergence. CASSANDRA uses randomness only as a tie-breaker among otherwise indistinguishable proposals; ordering is driven by the proposal priority rather than designated-leader or coin-driven leader election.

**Sharding Protocols.** Sharding protocols scale by splitting work across committees and optimizing cross-shard or geographic coordination [2, 24, 25, 32, 33, 53]. This direction is orthogonal: each shard still needs strong-quorum consensus internally, whereas CASSANDRA could serve as an intra-shard component that preserves recoverable local progress under weak-quorum connectivity.

**Concurrent and Multi-Leader BFT.** Concurrent and multi-leader protocols [7, 22, 29, 38–40, 47, 48] reduce the bottleneck of a single designated proposer under normal conditions. They improve steady-state throughput by letting multiple replicas inject work concurrently. Nevertheless, they still require global coordination and strong-quorum certificates to reconcile concurrent proposals into a final order. CASSANDRA adopts the multi-proposer intuition but separates weak-quorum recoverable ordering from strong-quorum finality, which is the key distinction under partitions.

**Asynchronous Protocols.** Asynchronous and DAG-based BFT protocols remove timing assumptions and decouple dissemination from ordering, with recent work also using trusted hardware for efficiency [14, 15, 31, 43, 51]. They still need classical quorum certificates among reachable replicas; CASSANDRA targets weak-quorum partitions by preserving recoverable progress with weaker certificates before full finality returns.

## 9 CONCLUSION

We presented CASSANDRA, a BFT protocol that breaks the all-or-nothing progress model of traditional consensus by enabling partial, safety-preserving progress even when no strong quorum is reachable. CASSANDRA achieves this through three key design ideas: a two-tier certification framework that decouples partial progress (via weak-quorum) from final commitment (via strong-quorum), a deterministic proposal-priority rule that allows any connected component of  $f+1$  correct replicas to select a non-predefined leader and advance ordering independently, and a decoupled pacemaker that drives round progression via certificates rather than synchronized timeouts. Together, these mechanisms enable isolated partitions to accumulate useful ordering progress that is implicitly reconciled once the network heals, eliminating the need for an explicit recovery protocol. Our evaluation shows that CASSANDRA remains competitive with state-of-the-art BFT protocols under stable conditions, while delivering continuous partial throughput during partitions and reducing wasted work upon reconnection.

## 10 ACKNOWLEDGEMENTS

This work is partially funded by NSF Award Number 2245373.

## REFERENCES

- [1] [n. d.]. Cassandra Source Code. <https://github.com/apache/incubator-resilientdb/tree/cassandra>.
- [2] Mustafa Al-Bassam, Alberto Sonnino, Shehar Bano, Dave Hrycyszyn, Sarah Meiklejohn, and George Danezis. 2018. Chainspace: A Sharded Smart Contracts Platform. In *NDSS*.
- [3] Ahmed Alquraan, Hatem Takturi, Mohammed Alfatafta, and Samer Al-Kiswany. 2018. An Analysis of Network-Partitioning Failures in Cloud Systems. In *USENIX OSDI*. 51–68.
- [4] Amazon Web Services. 2025. Summary of the Amazon DynamoDB Service Disruption in the Northern Virginia (US-EAST-1) Region. <https://aws.amazon.com/message/101925/>. (Accessed: 07-01-2026).
- [5] Apache ResilientDB. [n. d.]. Apache ResilientDB (Incubating). <https://resilientdb.apache.org/>.
- [6] Maria Apostolaki, Aviv Zohar, and Laurent Vanbever. 2017. Hijacking Bitcoin: Routing Attacks on Cryptocurrencies. In *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, 375–392. doi:10.1109/SP.2017.29
- [7] Balaji Arun and Binoy Ravindran. 2022. Scalable Byzantine Fault Tolerance via Partial Decentralization. In *PVLDB*.
- [8] Eric A. Brewer. 2000. Towards Robust Distributed Systems. In *ACM PODC*.
- [9] Miguel Castro and Barbara Liskov. 1999. Practical Byzantine Fault Tolerance. In *USENIX OSDI*. 173–186.
- [10] Miguel Castro and Barbara Liskov. 2002. Practical Byzantine Fault Tolerance and Proactive Recovery. In *ACM Transactions on Computer Systems*. 398–461.
- [11] Cloudflare. 2025. Cloudflare outage on December 5, 2025. <https://blog.cloudflare.com/5-december-2025-outage/>. Accessed: 07-01-2026.
- [12] Cloudflare. 2025. Cloudflare outage on November 18, 2025. <https://blog.cloudflare.com/18-november-2025-outage/>. Accessed: 07-01-2026.
- [13] Cryptonary. 2021. Solana Network Went Down Again; Mainnet Beta Restarts After 7 Hours of Outage. In *Online resource*.
- [14] George Danezis, Lefteris Kokoris-Kogias, Alberto Sonnino, and Alexander Spiegelman. 2022. Narwhal and Tusk: A DAG-Based Mempool and Efficient BFT Consensus. In *ACM EuroSys*.
- [15] Sisi Duan, Michael K. Reiter, and Haibin Zhang. 2018. BEAT: Asynchronous BFT Made Practical. In *ACM CCS*.
- [16] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer. 1988. Consensus in the Presence of Partial Synchrony. In *Journal of the ACM*. 288–323.
- [17] Rosario Gennaro, Stanislaw Jarecki, Hugo Krawczyk, and Tal Rabin. 1999. Secure Distributed Key Generation for Discrete-Log Based Cryptosystems. In *EUROCRYPT*. 295–310.
- [18] Seth Gilbert and Nancy Lynch. 2012. Perspectives on the CAP Theorem. In *IEEE Computer*.
- [19] Neil Girdharan, Florian Suri-Payer, Ittai Abraham, Lorenzo Alvisi, and Natacha Crooks. 2024. AutoBahn: Seamless High Speed BFT. In *ACM SOSP*.
- [20] Guy Golan Gueta, Ittai Abraham, Shelly Grossman, Dahlia Malkhi, Benny Pinkas, Michael K. Reiter, Dragos-Adrian Seredinschi, Orr Tamir, and Alin Tomescu. 2019. SBFT: A Scalable and Decentralized Trust Infrastructure. In *IEEE DSN*.
- [21] Suyash Gupta, Jelle Hellings, and Mohammad Sadoghi. 2021. Fault-Tolerant Distributed Transactions on Blockchain. In *Synthesis Lectures on Data Management*.
- [22] Suyash Gupta, Jelle Hellings, and Mohammad Sadoghi. 2021. RCC: Resilient Concurrent Consensus for High-Throughput Secure Transaction Processing. In *IEEE ICDE*.
- [23] Suyash Gupta, Sajjad Rahnama, Jelle Hellings, and Mohammad Sadoghi. 2020. ResilientDB: Global Scale Resilient Blockchain Fabric. In *PVLDB*. 868–883.
- [24] Jelle Hellings, Daniel P. Hughes, Joshua Primero, and Mohammad Sadoghi. 2023. Cerberus: Minimalistic Multi-Shard Byzantine-Resilient Transaction Processing. In *Journal of Systems Research*.
- [25] Jelle Hellings and Mohammad Sadoghi. 2021. ByShard: Sharding in a Byzantine Environment. In *PVLDB*.
- [26] Kendric Hood, Joseph Oglio, Mikhail Nesterenko, and Gokarna Sharma. 2021. Partitionable Asynchronous Cryptocurrency Blockchain. In *IEEE ICBC*.
- [27] Van Jacobson. 1988. Congestion Avoidance and Control. In *ACM SIGCOMM*.
- [28] Dakai Kang, Suyash Gupta, Dahlia Malkhi, and Mohammad Sadoghi. 2025. HotStuff-1: Linear Consensus with One-Phase Speculation. In *SIGMOD*.
- [29] Dakai Kang, Sajjad Rahnama, Jelle Hellings, and Mohammad Sadoghi. 2024. SpotLess: Concurrent Rotational Consensus Made Practical Through Rapid View Synchronization. In *IEEE ICDE*.
- [30] Jonathan Katz and Yehuda Lindell. 2014. Introduction to Modern Cryptography (2nd Edition). In *Chapman and Hall/CRC*.
- [31] Idit Keidar, Eleftherios Kokoris-Kogias, Oded Naor, and Alexander Spiegelman. 2021. All You Need is DAG. In *ACM PODC*.
- [32] S. Keshav, W. Golab, B. Wong, S. Rizvi, and S. Gorbunov. 2018. RCanopus: Making Canopus Resilient to Failures and Byzantine Faults. In *arXiv preprint arXiv:1810.09300*.
- [33] Eleftherios Kokoris-Kogias, Philipp Jovanovic, Linus Gasser, Nicolas Gailly, Ismail Khoffi, Lars M. Fischer, and Bryan Ford. 2018. OmniLedger: A Secure, Scale-Out, Decentralized Ledger via Sharding. In *IEEE S&P*.
- [34] Ramakrishna Kotla, Lorenzo Alvisi, Mike Dahlin, Allen Clement, and Edmund Wong. 2007. Zyzzyva: Speculative Byzantine Fault Tolerance. In *ACM SOSP*.
- [35] Andrew Lewis-Pye. 2022. Quadratic Worst-Case Message Complexity for State Machine Replication in the Partial Synchrony Model. In *arXiv preprint arXiv:2201.01107*.
- [36] Andrew Lewis-Pye and Ittai Abraham. 2023. Fever: Optimal Responsive View Synchronisation. In *arXiv preprint arXiv:2301.09881*.
- [37] Andrew Lewis-Pye, Dahlia Malkhi, Oded Naor, and Kartik Nayak. 2024. Lumiere: Making Optimal BFT for Partial Synchrony Practical. In *ACM PODC*.
- [38] Hanzheng Lyu, Shaokang Xie, Jianyu Niu, Ivan Beschastnikh, Yinqian Zhang, Mohammad Sadoghi, and Chen Feng. 2025. Orthus: Accelerating Multi-BFT Consensus Through Concurrent Partial Ordering of Transactions. In *IEEE ICDE*.
- [39] Hanzheng Lyu, Shaokang Xie, Jianyu Niu, Chen Feng, Yinqian Zhang, and Ivan Beschastnikh. 2025. Ladon: High-Performance Multi-BFT Consensus via Dynamic Global Ordering. In *ACM EuroSys*.
- [40] Hanzheng Lyu, Shaokang Xie, Jianyu Niu, Mohammad Sadoghi, Yinqian Zhang, Cong Wang, Ivan Beschastnikh, and Chen Feng. 2025. HYDRA: Breaking the Global Ordering Barrier in Multi-BFT Consensus. In *arXiv preprint arXiv:2511.05843*.
- [41] Dahlia Malkhi and Kartik Nayak. 2023. Hotstuff-2: Optimal two-phase responsive bft. In *Cryptology ePrint Archive, Paper 2023/397*.
- [42] Jean-Philippe Martin and Lorenzo Alvisi. 2005. Fast Byzantine Consensus. In *IEEE DSN*.
- [43] Andrew Miller, Yu Xia, Kyle Croman, Elaine Shi, and Dawn Song. 2016. The Honey Badger of BFT Protocols. In *ACM CCS*.
- [44] Torben P. Pedersen. 1991. A Threshold Cryptosystem without a Trusted Party. In *EUROCRYPT*. 522–526.
- [45] Mohammad Sadoghi. 2025. The Problems of Consensus: An Ethical Inquiry into Democratic and Decentralized Principles. In *SpringerBriefs in Philosophy*.
- [46] Alexander Spiegelman, Neil Girdharan, Alberto Sonnino, and Lefteris Kokoris-Kogias. 2022. Bullshark: DAG BFT Protocols Made Practical. In *ACM CCS*.
- [47] Chrysoula Stathakopoulou, Tudor David, Matej Pavlovic, and Marko Vukolic. 2022. Mir-BFT: Scalable and Robust BFT for Decentralized Networks. In *Journal of Systems Research*.
- [48] Chrysoula Stathakopoulou, Matej Pavlovic, and Marko Vukolic. 2022. State Machine Replication Scalability Made Simple. In *ACM EuroSys*.
- [49] Andrei Tonkikh, Balaji Arun, Zhuolun Xiang, Zekun Li, and Alexander Spiegelman. 2025. Raptr: Prefix Consensus for Robust High-Performance BFT. In *arXiv preprint arXiv:2504.18649*.
- [50] Shaokang Xie, Dakai Kang, Junchao Chen, Suyash Gupta, Daniel P. Hughes, and Mohammad Sadoghi. 2026. Cassandra: Consensus with Partial Progress via Robust Partitionable View Synchronization. arXiv:2607.02856 [cs.DC] <https://arxiv.org/abs/2607.02856>
- [51] Shaokang Xie, Dakai Kang, Hanzheng Lyu, Jianyu Niu, and Mohammad Sadoghi. 2025. Fides: Secure and Scalable Asynchronous DAG Consensus via Trusted Components. In *arXiv preprint arXiv:2501.01062*.
- [52] Maofan Yin, Dahlia Malkhi, Michael K. Reiter, Guy Golan Gueta, and Ittai Abraham. 2019. HotStuff: BFT Consensus with Linearity and Responsiveness. In *ACM PODC*.
- [53] Mahdi Zamani, Mahnush Movahedi, and Mariana Raykova. 2018. RapidChain: Scaling Blockchain via Full Sharding. In *ACM CCS*.

## A CORRECTNESS PROOF

This section provides the full proofs of Theorems 1, 2, and 3.

### A.1 Safety

We assume a Byzantine fault model with  $n = 3f+1$  replicas, where at most  $f$  replicas are Byzantine. A *strong quorum* requires  $Q = 2f+1$  votes, and a *weak quorum* requires  $Q_{\text{weak}} = f+1$  votes. We say two proposals *conflict* if neither is an ancestor of the other.

**LEMMA 1 (QUORUM INTERSECTION).** *Any two strong quorums of size  $2f+1$  intersect in at least  $f+1$  replicas; in particular, they share at least one correct replica.*

**PROOF.** Let  $Q_1$  and  $Q_2$  be two strong quorums with  $|Q_1| = |Q_2| = 2f+1$ . Since  $n = 3f+1$ ,

$$|Q_1 \cap Q_2| \geq |Q_1| + |Q_2| - n = (2f+1) + (2f+1) - (3f+1) = f+1.$$

As at most  $f$  replicas are Byzantine,  $Q_1 \cap Q_2$  contains at least one correct replica.  $\square$

**LEMMA 2 (NO DOUBLE VOTE).** *A correct replica votes for at most one proposal in a given round.*

**PROOF.** Correct replicas follow the protocol. In each round  $r$ , a correct replica selects a single candidate proposal according to the priority rule and broadcasts VOTE messages for exactly that proposal.  $\square$

**LEMMA 3 (PoR UNIQUENESS).** *For any round  $r$ , there cannot exist two valid PoR certificates that certify conflicting proposals.*

**PROOF.** Assume for contradiction that there exist two valid PoRs in round  $r$  certifying conflicting proposals  $b$  and  $b'$ . Each PoR requires  $2f+1$  votes. By Lemma 1, the two voter sets intersect in at least one correct replica. That replica would have voted for both  $b$  and  $b'$  in round  $r$ , contradicting Lemma 2.  $\square$

**LEMMA 4 (LOCK MONOTONICITY).** *For any correct replica  $p_i$ , its lock  $lock_i$  is monotonically non-decreasing in round number. Moreover, once  $p_i$  observes a PoR for a proposal, it locks that proposal and will not vote for any conflicting proposal unless it later observes a strictly higher-round PoR on the competing proposal's certified ancestry.*

**PROOF.** Whenever  $p_i$  observes a valid  $PoR_r$  for proposal  $b_r$ , it updates  $lock_i$  to  $b_r$  unless it is already locked on a descendant of  $b_r$ . Thus  $\text{round}(lock_i)$  can only increase over time.

After locking, SAFETOVOTE allows  $p_i$  to vote only for a proposal that either (i) extends the current locked branch, or (ii) carries a strictly higher-round PoR that justifies switching. By protocol validity, any such higher-round PoR must lie on the proposal's certified ancestry. Therefore,  $p_i$  never votes for a conflicting proposal unless it has first observed higher certified evidence that justifies the switch.  $\square$

**LEMMA 5 (COMMIT IMPLIES A LOCKED SET).** *If a proposal  $w = b_r$  is committed by the Two-PoR commit rule via a certified chain  $b_r \leftarrow b_{r+1}$  with PoRs  $PoR_r$  and  $PoR_{r+1}$ , then there exists a set  $L$  of at least  $f+1$  correct replicas such that every replica in  $L$  is locked on  $w$  (or its descendants).*

**PROOF.** Let  $Q_{r+1}$  be the voter set that formed  $PoR_{r+1}$  for  $b_{r+1}$ . Since  $|Q_{r+1}| = 2f+1$  and at most  $f$  replicas are Byzantine,  $Q_{r+1}$  contains at least  $f+1$  correct replicas; let  $L$  be those correct voters.

By protocol validity, a correct replica votes for  $b_{r+1}$  only after verifying that it extends  $b_r$  and that its certified justification includes the parent certificate  $PoR_r$  on  $b_r$  (carried either directly as the round-entry certificate or as the highest justified PoR on  $b_{r+1}$ 's certified ancestry). Hence, every replica in  $L$  has observed  $PoR_r$  before voting for  $b_{r+1}$ . By Lemma 4, each such replica locks on  $w = b_r$  (or a descendant on the same branch).  $\square$

**Theorem 1 (Safety).** *No two correct replicas commit (finalize) conflicting proposals.*

**PROOF OF THEOREM 1.** Assume for contradiction that two correct replicas commit conflicting proposals  $w$  and  $b$ , with  $\text{round}(w) < \text{round}(b)$ .

Since  $w$  is committed, by Lemma 5, there exists a set  $L$  of at least  $f+1$  correct replicas locked on  $w$  (or its descendants).

Since  $b$  is committed, there must exist at least one PoR that certifies a proposal conflicting with  $w$  in some round after  $\text{round}(w)$ . Let  $PoR^*$  be the earliest such PoR, certifying a proposal  $x^*$  that conflicts with  $w$ , and let  $Q^*$  be its voter set.

Because  $|Q^*| = 2f+1$  and  $|L| = f+1$ , we have

$$|Q^* \cap L| \geq (2f+1) + (f+1) - (3f+1) = 1.$$

Thus,  $Q^*$  intersects  $L$  in at least one correct replica  $p$ .

Replica  $p$  is locked on the branch of  $w$ . For  $p$  to vote for  $x^*$ , SAFETOVOTE requires that  $x^*$  either extends  $p$ 's locked branch or carries a strictly higher-round PoR on  $x^*$ 's certified ancestry that justifies switching. However,  $x^*$  conflicts with  $w$ , so it does not extend the locked branch. Moreover, any such higher-round PoR on  $x^*$ 's ancestry would itself certify a conflicting proposal earlier than  $PoR^*$ , contradicting the choice of  $PoR^*$  as the earliest conflicting PoR after  $w$ . Therefore,  $p$  cannot vote for  $x^*$ , contradicting  $p \in Q^*$ .

Hence, no two correct replicas can commit conflicting proposals.  $\square$

### A.2 Liveness

**Definition 1 (Timeout-Sufficient Replica).** *Fix a stabilized synchronous interval whose actual message delay bound is  $\Delta$ . A correct replica  $p_i$  is timeout-sufficient during that interval if its local timeout baseline satisfies  $\delta_i \geq \Delta$ .*

**LEMMA 6 (EVENTUAL TIMEOUT SUFFICIENCY).** *During any sufficiently long stabilized synchronous interval in which at least  $2f+1$  correct replicas are mutually connected, every correct replica eventually becomes timeout-sufficient.*

**PROOF.** Fix such a stabilized synchronous interval with actual delay bound  $\Delta$ . Consider any correct replica  $p_i$ .

If  $p_i$  cannot yet collect enough  $\langle \text{SYNC-READY}, v \rangle$  messages to locally form a synchronization certificate, it keeps retransmitting its  $\langle \text{SYNC-READY}, v \rangle$  message. Since at least  $2f+1$  correct replicas are mutually connected and all messages among them are delivered within  $\Delta$ , eventually every correct replica receives  $2f+1$  valid SYNCREADY messages for some synchronization view and forms a local  $\langle \text{SYNC-CERT}, v \rangle$ .

If replica  $p_i$  locally forms a synchronization certificate but does not receive certificates from at least  $f+1$  distinct replicas within time  $\delta_i$ , it doubles its  $\delta_i$  and retries in a higher synchronization view. Hence, as long as  $\delta_i < \Delta$ , repeated failures cause  $\delta_i$  to grow exponentially.

Once  $\delta_i \geq \Delta$ , after  $p_i$  forms its local synchronization certificate, every correct replica that is already in the same synchronization view can form and broadcast its own synchronization certificate within at most  $\Delta$ , and that certificate is delivered back to  $p_i$  within at most another  $\Delta$ . Therefore, replica  $p_i$  receives remote synchronization certificates from at least  $f+1$  distinct replicas within time  $\delta_i$ , so the calibration attempt succeeds.

Thus, after finitely many retries, every correct replica reaches a timeout baseline  $\delta_i \geq \Delta$  and remains timeout-sufficient throughout the stabilized interval.  $\square$

**Definition 2** (Good round). *A round  $r$  is good if there exists a stabilized synchronous interval of length  $\geq 3\Delta$ , partitioned into three consecutive subintervals  $I_1$ ,  $I_2$ , and  $I_3$  each of length  $\geq \Delta$ , such that: (i) by the end of  $I_1$ , all correct replicas have entered round  $r$  and are timeout-sufficient; (ii) during  $I_1 \cup I_2$ , every proposal broadcast by a correct replica in round  $r$  is delivered to every correct replica; and (iii) during  $I_3$ , all correct replicas remain in round  $r$ , and every vote broadcast by a correct replica in round  $r$  is delivered to every correct replica.*

**LEMMA 7** (GOOD ROUND EXISTS). *During any sufficiently long stabilized synchronous period, the pacemaker ensures that all correct replicas eventually synchronize their round counters and enter good rounds.*

**PROOF.** Consider a stabilized synchronous period in which messages between correct replicas are delivered within  $\Delta$ . By Lemma 6, after finitely many retries, every correct replica becomes timeout-sufficient. If any correct replica enters a higher round using a valid PoR or RC, it broadcasts that certificate. If a lagging correct replica has not yet seen the standalone certificate, it will still receive a valid higher-round proposal carrying the same certificate as its *enterCert*. In either case, within at most one message delay, every correct replica receives round-entry evidence for that higher round and can catch up.

Moreover, when a lagging replica catches up to an already active round, it joins that round using only the remaining local waiting needed before entering the election phase, rather than restarting a full fresh round. Thus, during a sufficiently long stabilized synchronous period, the skew among correct replicas is continuously reduced, and there exists a round  $r$  in which all correct replicas enter early enough to complete the proposal-exchange window ( $> 2\Delta$ ) and the election window ( $> \Delta$ ) together. Hence, round  $r$  is good by Definition 2.

As the stabilized synchronous period continues, the same argument applies repeatedly, so good rounds recur.  $\square$

**LEMMA 8** (PoR IN A GOOD ROUND (PROBABILISTIC)). *In any good round  $r$ , honest replicas form a valid PoR for a single proposal with probability at least  $\frac{2f+1}{n}$ .*

**PROOF.** Fix a good round  $r$ . By Definition 2, by the end of the proposal-exchange window ( $I_1 \cup I_2$ ), every correct replica has received every proposal broadcast by a correct replica in round  $r$ . We assume that each replica accepts at most one proposal per author per round, ignoring additional proposals from the same author as equivocations. Hence, the number of candidate proposals considered in round  $r$  is at most  $n$ .

Let  $\mathcal{M}_r$  denote the set of proposals with maximum log strength under  $\mathcal{S}(\cdot)$  among the proposals received by a correct replica in round  $r$ . If  $|\mathcal{M}_r| = 1$ , all correct replicas select the same strongest proposal and vote for it.

Otherwise, ties are broken using the common coin  $C_r$  for round  $r$ . Since  $C_r$  is obtained by threshold aggregation of  $f+1$  signature shares from  $f+1$  distinct replicas, all correct replicas compute the same  $C_r$ . For each proposal  $\mathcal{P} \in \mathcal{M}_r$ , replicas compute

$$\text{Score}(\mathcal{P}) = H(C_r \parallel \mathcal{P}.\text{proposer\_id})$$

and select the proposal with the highest score.

Consider the event that the highest score among the tied proposers corresponds to an honest proposer. Since proposer identities are fixed and  $C_r$  is unpredictable to the adversary before round  $r$ , the top-scoring proposer is uniformly distributed over the  $n$  proposers, and is honest with probability  $\frac{2f+1}{n}$ . Conditioned on this event, all correct replicas select the same honest proposal.

During the election window  $I_3$ , all correct replicas broadcast their VOTES, and by Definition 2, those votes are delivered to every correct replica within one message delay. Therefore, the selected proposal receives at least  $2f+1$  votes, allowing correct replicas to assemble a valid PoR in round  $r$ .  $\square$

**Theorem 2** (Liveness (with probability 1)). *During any sufficiently long stabilized synchronous period, correct replicas commit new proposals continuously; over an unbounded sequence of such stabilized periods, commits occur infinitely often with probability 1.*

**PROOF OF THEOREM 2.** Consider a sufficiently long stabilized synchronous period. By Lemma 6, sufficiently long stabilized synchronous periods first make all correct replicas timeout-sufficient; then, by Lemma 7, good rounds recur. In each good round, Lemma 8 implies that a PoR is formed with non-zero probability.

Assuming fresh common-coin outputs across rounds, these per-round success events occur with a fixed non-zero probability, so over an unbounded sequence of good rounds, PoRs occur infinitely often with probability 1.

Whenever two consecutive rounds produce PoRs for proposals that extend one another, the Two-PoR commit rule commits the earlier proposal in the two-chain. Hence, during any sufficiently long stabilized synchronous period, correct replicas commit new proposals continuously; over an unbounded sequence of such stabilized periods, commits occur infinitely often with probability 1.  $\square$

### A.3 Partial Liveness

CASSANDRA distinguishes between *commitment* (which requires strong quorums and PoRs) and *partial progress* (which preserves useful work and availability evidence during partitions).

**LEMMA 9** (WEAK QUORUM PROGRESS). *In any connected component that contains at least  $Q_{\text{weak}} = f+1$  honest replicas, whose*

internal message delays are bounded by  $\Delta$ , and in which all participating correct replicas are timeout-sufficient (i.e., they have local timeout baselines satisfying  $\delta_i \geq \Delta$ ), the component can form at least one PoA in a round.

**PROOF.** Let  $C$  be a connected component containing  $k \geq f+1$  honest and timeout-sufficient replicas. Within  $C$ , all honest replicas receive each other's proposals within the proposal-exchange window ( $> 2\Delta$ ). By applying the deterministic *Priority Rule*, these replicas converge on the same strongest candidate and broadcast votes for it during the subsequent election window ( $> \Delta$ ). Since  $|C| \geq Q_{\text{weak}} = f+1$ , replicas are guaranteed to aggregate the required weak quorum of votes. Consequently, replicas in  $C$  can locally assemble a valid PoA.  $\square$

**LEMMA 10 (RC PROGRESS IN A WEAK COMPONENT).** *In any connected component that contains at least  $Q_{\text{weak}} = f+1$  honest replicas, the component can form a valid RC for round  $r+1$ .*

**PROOF.** If no PoR is formed in round  $r$ , then every correct replica in the component eventually reaches the election timeout for round  $r$  and broadcasts a WISHPROPOSAL( $r$ ) message. Since the component contains at least  $f+1$  honest replicas and those messages are eventually delivered (without strict timing guarantees), replicas in the component can collect at least  $f+1$  such messages. By the protocol rule for round synchronization, these messages form a valid RC for round  $r+1$ .  $\square$

**LEMMA 11 (DATA AVAILABILITY).** *If a proposal is certified by a PoA or a PoR, its data is available at at least one correct replica. Hence, any correct replica can lazily retrieve it via the catch-up mechanism when connectivity permits.*

**PROOF.** A PoA or a PoR requires at least  $f+1$  votes, which guarantees that at least one correct replica has verified and stored the proposal. Hence, this certificate ensures that the proposal remains retrievable in the system.  $\square$

**Theorem 3 (Partial Liveness).** *CASSANDRA satisfies the **Partial Liveness** property. Specifically, during network partitions, any connected component that contains at least  $f+1$  correct replicas and whose internal message delays are bounded by  $\Delta$ , and whose correct replicas use local timeout baselines satisfying  $\delta_i \geq \Delta$ , continues to advance rounds locally and produce PoA-certified partial progress. After network recovery, this accumulated partial progress remains recoverable and can be incorporated into subsequent PoR-backed commitment.*

**PROOF OF THEOREM 3.** Let  $C$  be a connected component containing at least  $f+1$  correct and timeout-sufficient replicas and timely internal communication.

By Lemma 9, in each local round, replicas in  $C$  can form a PoA for the strongest locally visible proposal, thereby preserving certified partial progress. If the component cannot form a strong certificate in that round, then by Lemma 10, the same replicas can still form an RC for the next round and continue advancing locally. Thus, even without a global strong quorum,  $C$  continues both to make round progress and to accumulate recoverable PoA-certified work.

A PoA does not imply commitment, but by Lemma 11, it guarantees that the certified proposal remains available at some correct replica. Once global connectivity is restored and a strong quorum

$Q = 2f+1$  becomes reachable again, replicas exchange their local histories, compare the available branches via the deterministic priority rule, and resume forming PoRs on the strongest recoverable branch. The catch-up and back-fill mechanisms then retrieve and extend previously accumulated PoA-backed proposals on that branch, allowing the system to reincorporate partition-time partial progress into subsequent PoR-backed commitment.

Therefore, CASSANDRA satisfies Partial Liveness.  $\square$

## B RELATED WORK

The design of CASSANDRA draws inspiration from and builds upon several decades of research in BFT consensus. We categorize the most relevant literature into four primary domains: partition-tolerant consensus, sharding protocols, asynchronous BFT, and view synchronization.

**Network Partition.** The challenge of maintaining consistency and availability under network partitions is fundamentally tied to the CAP theorem. Traditional BFT protocols typically prioritize safety and stall indefinitely when a strong quorum ( $2f+1$ ) cannot be assembled. *Partitionable Blockchain Consensus* [26] was among the first to formalize this problem. It proves impossibility in a fully asynchronous model, then leverages external *partition detectors* to coordinate the partition. Raptr [49] introduces prefix consensus: replicas maintain safety by finalizing only the common transaction prefix. Then it reconciles divergent suffixes upon restoration. More recently, AutoBahn [19] preserves a limited form of partial progress under network disruptions, but this progress is largely confined to the dissemination layer rather than the ordering layer, and thus does not directly address *designated-leader dependence*.

In contrast to these approaches, CASSANDRA neither relies on external partition detectors nor confines partial progress to data dissemination alone. Instead, its core ordering path allows partitions to independently order transactions that are later integrated through an implicit merge method.

**View Synchronization.** View synchronization is fundamental for aligning replicas within a common round to ensure sustained consensus progress. While classic protocols like PBFT [9] utilize an exponential back-off strategy for eventual convergence, HotStuff [52] decouples this logic into a dedicated *pacemaker*. Building on this modularity, state-of-the-art mechanisms have evolved to address the quadratic communication overhead inherent in traditional designs. Notable advancements include Lewis-Pye's hierarchical, epoch-based framework [35, 36], which achieves amortized  $O(n)$  communication complexity, and Lumiere [37], which further optimizes steady-state performance with optimistic responsiveness.

While these advancements significantly enhance steady-state performance, they remain fundamentally dependent on a  $2f+1$  strong quorum of "new-view" messages to transition between rounds, leading to total system stalls during network partitions. Cassandra fills this operational void by introducing a novel pacemaker that facilitates round advancement through the evidence from  $f+1$  replicas, thereby sustaining partial progress where other state-of-the-art pacemakers fail to coordinate.

**Dual-Path Design BFT.** A common theme in practical BFT systems is to expose an *optimistic fast path* that achieves low latency and

linear communication when the network is stable and the active leader is responsive, while retaining a *fallback path* that preserves safety under faults and delays.

Prior dual-path BFT protocols [20, 34, 42] achieve low latency via an optimistic fast path but typically remain leader-driven in the common case and rely on leader-based recovery to fall back under adverse conditions. DAG-based designs such as Bullshark [46] similarly incorporate a predefined leader in the synchronous path, while relying on a random coin to select a fallback leader for global convergence. In contrast, although CASSANDRA also employs a coin, it is used only as a tie-breaker when multiple proposals remain indistinguishable under the deterministic priority rule, rather than as a mechanism for leader election or round synchronization. As a result, CASSANDRA removes designated-leader (and coin-driven leader-election) dependence from its core ordering path via partitionable leader selection and deterministic proposal priority, enabling components to continue making order-relevant partial progress even when global leader reachability or coin-driven coordination becomes ineffective under partitions.

**Sharding Protocols.** Sharded systems aim to improve scalability by partitioning replicas into smaller committees that process disjoint subsets of state or transactions in parallel. Early systems such as Chainspace [2] explored object-centric sharding with BFT committees and atomic object locking. Building on this, OmniLedger [33] utilized secure randomness for epoch-based committee assignment, while RapidChain [53] pipelines block production across shards. Cerberus [24] further evolved this by employing per-shard DAGs to eliminate single-leader bottlenecks. Extending this line of work, ByShard [25] optimizes this paradigm by introducing a sharded database framework that efficiently integrates BFT consensus with atomic cross-shard coordination. RCanopus [32] geographically divides replicas into Byzantine Groups (BGs) that run local BFT, while a super-consensus orders the BG outputs.

While these systems significantly improve horizontal scalability, they still inherently stall if fewer than  $2f+1$  replicas remain in a shard after a partition. CASSANDRA's partial-progress technique is *orthogonal*. It could serve as an intra-shard consensus component, allowing each committee to process local transactions with non-zero throughput even when a partition leaves only  $f+1$  honest replicas connected.

**Concurrent and Multi-Leader BFT.** Recent work [7, 22, 29, 38–40, 47, 48] has also explored allowing multiple replicas to concurrently drive consensus, which helps reduce the bottleneck of a single designated leader under stable conditions. At a high level, this design may appear to mitigate *leader dependence*, since progress is no longer tied to one pre-assigned proposer. However, these protocols still require global coordination to reconcile concurrent proposals and remain constrained by the *strong quorum dependence*. While they improve steady-state throughput, they do not directly address the partition setting considered in CASSANDRA: failing to make useful progress during network partitions.

**Asynchronous Protocols.** Fully asynchronous protocols, such as HoneyBadgerBFT [43] and BEAT [15], eliminate timing assumptions by combining reliable broadcast with threshold cryptography. Recent advancements in DAG-based BFT, including DAG-Rider [31]

and Tusk [14], further optimize performance by decoupling data dissemination from ordering, achieving optimal resilience and amortized  $O(n^2)$  complexity. Fides [51] further enhances this model by offloading critical operations into trusted hardware for an optimized threshold and better performance.

Despite their resilience to asynchrony, these protocols remain bound by the *Strong Quorum Dependence*: none can make progress once fewer than a quorum of replicas are reachable, since they still require a classical quorum to form certificates. In contrast, CASSANDRA addresses this by allowing a weaker quorum to construct weaker certificates, achieving non-zero progress during weak quorums—a breakthrough from 0 to 1.